

Assignment 3: Agentic Shell (`doit`)

Yuval Reuveni & Arbel Tepper

August 2026

Contents

1	Introduction & System Overview	4
1.1	The Tool Set	4
1.2	State Layout	5
1.3	Context Assembly and Code Mapping	5
2	Single command at a time.	6
2.1	Implementation Details	6
2.2	Design Decisions	6
2.3	Example Interactions	7
2.4	Raw LPU Exchange (native tool-calling)	7
2.5	Limitations	8
2.6	ACDL Context Specification: DoitAgentSingleCommand	8
3	Identifying dangerous commands	9
3.1	Implementation Details	9
3.2	Design Decisions	10
3.3	Example Interactions	10
3.4	Guard Override in Practice	10
3.5	Limitations	11
3.6	ACDL Context Specification: DoitAgentSafety	11
4	Model flexibility	12
4.1	Implementation Details	12
4.2	Design Decisions	12
4.3	Evaluation of Local Models	12
4.4	Analysis of Divergences and Failure Modes	12
4.4.1	Gemma-3 Impossible Request Failure	12
4.4.2	Prompted Adapter Recovery Logs	13
4.5	Case-by-Case Results	13
4.5.1	Logged Reliability Numbers	14
4.6	Local-vs-Local Head-to-Head: qwen3 vs gemma3	14
4.7	Limitations	15
4.8	ACDL Context Specification: DoitAgentPrompted	15
5	Multi-turn	16
5.1	Implementation Details	16
5.2	Design Decisions	16
5.3	Example Multi-Turn Interactions	17
5.4	Limitations	17
5.5	ACDL Context Specification: DoitAgentHistory	17
6	Clarifications	19
6.1	Implementation Details	19
6.2	Design Decisions	19
6.3	Non-Annoyance Cap	19
6.4	Example Interaction	19
6.5	Limitations	20
6.6	ACDL Context Specification: DoitAgentClarify	20

7	Richer interactions	22
7.1	Implementation Details	22
7.2	Example Interaction	23
7.3	What Worked and What Didn't: the Repeated-Question Bug	23
7.4	Limitations	23
7.5	ACDL Context	24
8	Memory	24
8.1	Implementation Details	24
8.2	Design Decisions	24
8.3	Example Interaction	24
8.4	What Worked and What Didn't: a Memory Edit that Crashed	25
8.5	Limitations	26
8.6	ACDL Context Specification: DoitAgentMemory	26
9	Changing directories (change_dir)	28
10	User awareness	31
10.1	Implementation Details	31
10.2	Design Decisions	32
10.3	Example Interaction	32
10.4	Limitations	33
10.5	ACDL Context Specification: DoitAgentUserAware	33
11	Output awareness	35
11.1	Implementation Details	36
11.2	Design Decisions	36
11.3	Example Interaction	36
11.4	Limitations	37
11.5	ACDL Context	37
12	Multi-tasking	37
12.1	Implementation Details	37
12.2	Design Decisions	38
12.3	Example Interaction	38
12.4	Limitations	39
12.5	ACDL Context Specification: DoitAgentMultitask	39
13	Further extensions	42
13.1	Description of Three Extensions	42
13.2	Implementation of Multi-Step Command Plans & Retry	42
13.3	Design Decisions	42
13.4	Example Interaction	43
13.5	What Worked and What Didn't: a Plan that Stopped Halfway	43
13.6	Limitations	44
13.7	ACDL Context Specification: DoitAgentPlans	44
14	Conclusion	46

A Prompts and Tool Schemas	47
A.1 System Prompt: Base and Per-Version Deltas	47
A.2 Full System Prompt (final, v9)	50
A.3 Context Block Templates	52
A.4 Tool Schemas	52

1 Introduction & System Overview

This report documents our design and implementation of the `doit` agentic shell, a command-line utility that translates natural language instructions into shell commands and executes them on behalf of the user.

Following the core architecture of LLM systems, we establish a strict boundary between the **Language Processing Unit (LPU)**—responsible solely for generating structured tool decisions from context—and the **Controller**—a deterministic Python runtime environment that manages state, enforces safety checks, and handles command execution. This boundary is illustrated in Figure 1.

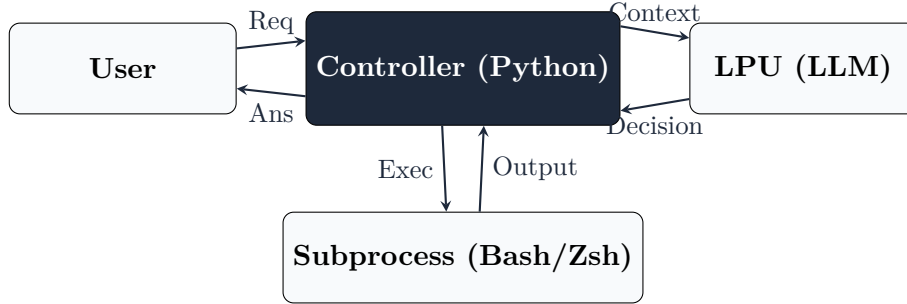


Figure 1: The `doit` architecture: the Controller wraps the LPU. The LPU only emits structured decisions from context, while the Controller owns all state, safety, and execution — it is the only component that touches the shell or replies to the user.

Rather than defining a tool per shell command (e.g., `ls` or `grep`), our agent exposes a small, unified set of universal tools — the shell string handed to `run_command` is itself the one command tool. The core set (`run_command`, `answer`, `ask_user`, `remember/forget`, `change_dir`) grows to eight as later phases add `read_session` and `plan`. We structure this report to follow the exact sections defined in the assignment specifications, detailing the design decisions, implementation details, log evidence, and limitations encountered for each module.

1.1 The Tool Set

Every decision the LPU makes is one call to one of the eight tools in Table 1. This fixed set *is* the decision schema — the single contract every supported model codes against. The complete argument-level schemas (`TOOL_SCHEMAS`) are in Appendix A.4.

Table 1: The eight tools that make up the agent’s entire decision schema.

Tool	Required args	Purpose
run_command	command, is_destructive, explanation	Translate and run one shell command; <code>is_destructive</code> is safety layer 1.
answer	text	Reply in plain text; also the finish signal that ends the turn.
ask_user	question (options optional)	Ask one clarifying question, answered within the same turn.
remember	fact	Store a durable fact/preference about the user.
forget	id	Delete a stored fact by its [id].
change_dir	path	Change the shell’s directory via the wrapper (the <code>cd-trap</code> fix).
read_session	session_id	Fetch another terminal session’s full history on demand.
plan	steps	Announce a multi-step plan (offered only when <code>enable_plans</code> is set).

1.2 State Layout

All runtime state lives under `~/.doit/`, keyed by the per-terminal `DOIT_SESSION` id, so concurrent windows stay isolated (Section 12):

```
~/.doit/
sessions/<id>.jsonl  # all runtime state, keyed by DOIT_SESSION
memories.json       # one JSON record per turn (request, steps, outputs, answer)
shell_hist/<id>      # global [{id, ts, text}] facts, shared across all sessions
cd_target_<id>       # ts|cwd|cmd lines from the shell hook (user awareness)
logs/               # change_dir hand-off file, read by the shell wrapper
                   # full raw LLM requests/responses (report evidence)
```

Listing 1: Runtime state under `~/.doit/`.

1.3 Context Assembly and Code Mapping

Each context block in our ACDL schemas is built by one function in `doitlib/context.py`, and `build_messages()` assembles them in a fixed order:

- The single `system` message — `AGENT_INSTRUCTIONS`, `SAFETY_INSTRUCTIONS`, and the per-feature `*_INSTRUCTIONS` blocks — $\longleftrightarrow$ `agent_instructions()`
- `ENV_INFO` (`env.cwd`, `env.shell`, `env.os`, `env.datetime`) $\longleftrightarrow$ `environment_block(config)`
- `sys.memories` $\longleftrightarrow$ `memory_block()`
- `sys.other_sessions` $\longleftrightarrow$ `other_sessions_block(session_id)`
- `sys.history` (replayed past turns) $\longleftrightarrow$ `history_messages(session_id)`
- `env.recent_user_commands` $\longleftrightarrow$ `user_shell_history_block(session_id)`
- `env.user_request` $\longleftrightarrow$ `user_request(request)`

The order matters: this session’s own history sits just above the request, so a follow-up like “sort them” resolves locally rather than against another window (Section 12). Figure 2 illustrates the assembly.

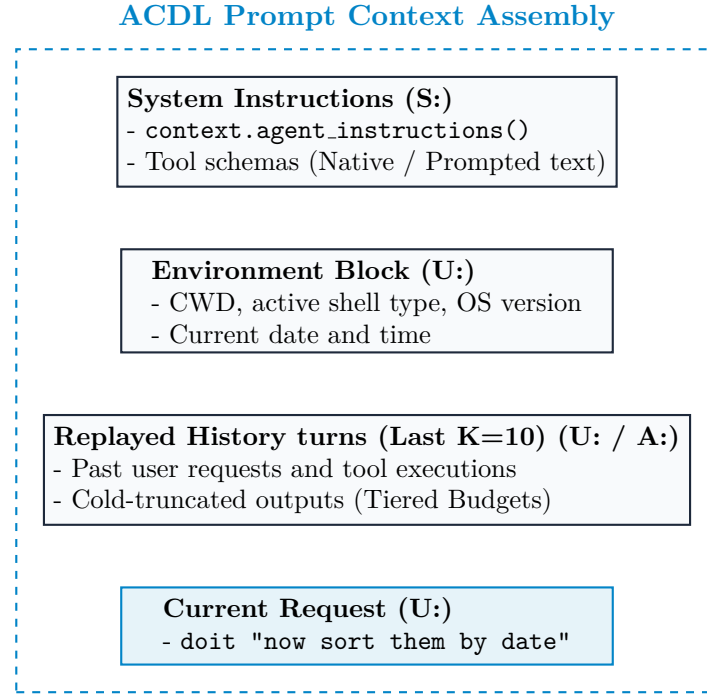


Figure 2: LPU input context block mapping.

2 Single command at a time.

The core requirement of this section is translating a single natural language request into a shell command, printing it, executing it in the user’s active shell, and displaying the output.

2.1 Implementation Details

We developed the entry-point executable `doit` in Python (placed on the user’s `PATH` via `~/.local/bin`). The system operates by assembling a system prompt, appending an environment block detailing the user’s shell parameters, and calling the LPU with a command-step cap of `max_steps=1`.

If the user’s prompt represents a command, the LPU calls the `run_command` tool, which is intercepted and executed via Python’s `subprocess.run`. If the prompt represents a general inquiry, the LPU responds with the `answer` tool.

2.2 Design Decisions

Shell-Function Wrapper for Directory Navigation (`cd`)

Problem: When a user runs a Python script, it runs in a child subprocess. Any directory change (`cd`) executed by the script is lost when the script exits, returning the user to the original terminal directory.

Solution: We resolved this by writing a dual-shell wrapper. The `change_dir` tool verifies that the path exists and writes it to a temporary file: `~/.doit/cd_target_${DOIT_SESSION}`. Upon `doit`’s termination, a shell function wrapper intercepts the exit, checks for the presence of this file, runs the native `cd` command inside the parent shell, and deletes the file. The `change_dir` tool and shell snippet that implement this are covered in full in Section 9.

Dual-Shell Variable Normalization

Context: Since different team members use different operating systems and shells (e.g., macOS/Zsh vs. Linux/Bash), command arguments and behavior can diverge.

Solution: The Controller auto-detects the host shell via `$SHELL` and includes the shell type and operating system in the environment prompt block. The system prompt instructs the LPU to emit syntax tailored to the host environment.

Primary API Model Selection

Choice: We selected `openai/gpt-4o-mini` as our primary API model. It is inexpensive, runs with low latency, and features robust native tool-calling capabilities that simplify core development.

2.3 Example Interactions

Below are two successful single-command interactions run using our primary model:

```
1 $ doit "show me all files here, including hidden ones"
2 $ ls -la
3 total 232
4 drwxr-xr-x@ 6 yuval  staff    192 Jul  7 18:23 photos
5 -rw-r--r--@ 1 yuval  staff  12405 Jul  7 18:24 notes.txt
6 -rw-r--r--@ 1 yuval  staff  30675 Jul  7 18:57 report.txt
7 -rwxr-xr-x@ 1 yuval  staff   1319 Jul  7 15:00 backup.sh
```

Listing 2: Listing files including hidden ones.

```
1 $ doit "how much disk space is left?"
2 $ df -h
3 Filesystem      Size   Used  Avail Capacity  iused ifree %iused  Mounted
   on
4 /dev/disk3s1    926Gi   17Gi   702Gi     3%    459k  4.3G     0%    /
```

Listing 3: Disk space command execution.

Not every request maps to a command. A physically impossible request is declined with the `answer` tool, and an off-topic request receives a short in-role refusal rather than compliance — in both cases nothing runs (`logs/phase1/p1demo.jsonl`):

```
1 $ doit "make my laptop fly"
2 I'm a shell command agent and can't help with that!
3
4 $ doit "tell me a joke"
5 I'm a shell command agent and can't help with jokes, but I'm here to
6 assist you with commands and tasks!
```

Listing 4: Impossible and off-topic requests: the model answers instead of running a command.

2.4 Raw LPU Exchange (native tool-calling)

The decision the model returns is a *structured tool call*, not free text. Below is the real request/response behind the first example, trimmed to essentials (full traffic in `logs/phase1/llm_raw.p1demo.jsonl`). The tool schemas travel out-of-band in the request's `tools` array (Appendix A.4); the model answers with a `tool_calls` object whose `arguments` is the JSON the Controller then executes.


```
// --- request (messages trimmed; full system prompt in Appendix A) ---
{
  "model": "openai/gpt-4o-mini",
  "messages": [
    {"role": "system", "content": "You are doit, ... reply with exactly one tool call ..."},
    {"role": "user", "content": "Environment:\n- current directory: ...\n- shell: /bin /zsh\n- OS: Darwin (arm64)"},
    {"role": "user", "content": "show me all files here, including hidden ones"}
  ],
  "tools": [ {"type": "function", "function": {"name": "run_command", ...}}, ... ] // schemas out-of-band
}

// --- response ---
{
  "choices": [{
    "finish_reason": "tool_calls",
    "message": {
      "role": "assistant",
      "content": null,
      "tool_calls": [{
        "type": "function",
        "id": "call_kZJpZjMreuLpguzwwLYZ7t0q",
        "function": {
          "name": "run_command",
          "arguments": "{\"command\":\"ls -la\", \"is_destructive\":false, \"explanation\": \"Lists all files in the current directory, including hidden ones.\"}"
        }
      }]
    }
  }],
  "usage": {"prompt_tokens": 527, "completion_tokens": 38, "total_tokens": 565}
}
```

Listing 5: Trimmed raw request/response for doit "show me all files here, including hidden ones" (openai/gpt-4o-mini).

2.5 Limitations

Single-command mode is deliberately minimal, and that shows in three ways. First, the turn runs exactly one command (`max_steps=1`), so a request that genuinely needs several steps cannot be handled at this stage — this is what the Phase 9 `plan` tool (Section 13) later lifts. Second, the command is executed straight from a single model call, and its output is never fed back, so the model cannot see the result and self-correct here (added later in Section 11); nothing checks that the command does what the user *meant*, only that it is not destructive. Third, correctness depends on the model emitting syntax that fits the host shell and OS — weaker models sometimes emit GNU-only flags that fail on BSD/macOS (Section 4).

2.6 ACDL Context Specification: DoitAgentSingleCommand

The diagram below shows the context we build for a single-command turn: the system prompt, the environment block, and the user request. In single-command mode (`max_steps=1`) the model makes one decision and the turn ends. The command's output is printed and logged, but it is never fed back to the model, so the diagram stops at the request.

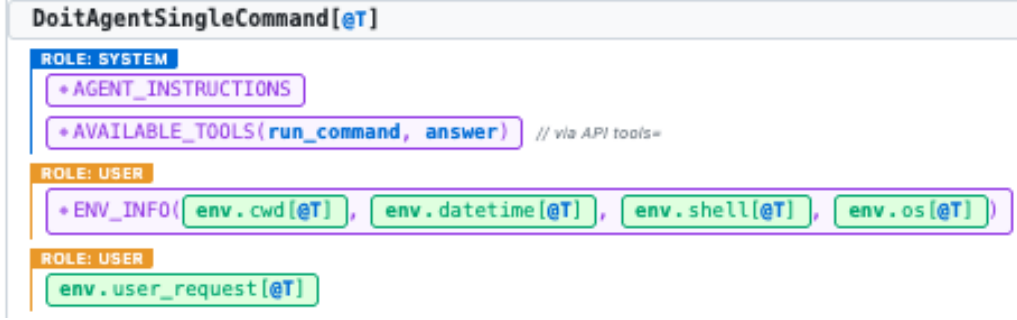


Figure 3: Visual representation of the ACDL context for DoitAgentSingleCommand.

```

// doit v1---single-command agent (Phase 1)
// The context we assemble and send: system + env + request. In single-command
// mode the decision and its result are output, never fed back, so neither
// appears. Native tools travel out-of-band via the API tools= parameter.

DoitAgentSingleCommand[@T]: {
  S: {
    AGENT_INSTRUCTIONS
    AVAILABLE_TOOLS(run_command, answer) // via API tools=
  }
  U: {
    ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
  }
  U: env.user_request[@T]
}

```

Listing 6: ACDL source for DoitAgentSingleCommand (acd1/v1.single.acd1).

3 Identifying dangerous commands

To prevent the agent from accidentally corrupting the host machine or executing destructive actions without oversight, we implement safety policies that intercept filesystem mutations.

3.1 Implementation Details

Safety is handled through a double-layer verification policy (defense in depth):

1. **LPU Self-Report (Layer 1):** The `run_command` tool schema requires the LPU to populate the `is_destructive` boolean flag and provide an `explanation` of what the command will do.
2. **Deterministic Guard (Layer 2):** The Controller intercepts the decision and passes the command through a pre-compiled regex guard in `safety.py`. If the regex matches mutating operations (e.g. `rm`, `mv`, `cp`, `mkdir`, redirects `>`, `>>`, `git reset`), it overrides a false “safe” report, forcing `is_destructive=True`.

3.2 Design Decisions

In-Band Self-Report vs. Separate LLM Judge Call

Alternative Rejected: We rejected using a separate LLM call to classify command destructiveness.

Rationale: A second LLM call doubles latency and API costs. Because the regex guard is required to handle safety deterministically anyway, we require the LPU to report destructiveness in the same JSON object as the command itself. The regex guard acts as the fallback mechanism. If the guard overrides the model's safety flag, the event is logged for analysis.

Safety Refusals vs. Confirmation Gates

Hard Refusals: Any command containing `sudo` or starting with an interactive full-screen program (e.g., `vim`, `nano`, `less`, `top`) is blocked immediately in the Controller. No confirmation is offered.

Confirmation Gates: When a command is flagged destructive (either by the model or the regex override), the Controller halts, displays the command and its explanation, and prompts the user with `Proceed? [y/N]`. Execution only proceeds if the user inputs `y` or `yes`.

3.3 Example Interactions

```

1 $ doit "delete junk.txt"
2 [WARNING] This command modifies the filesystem:
3     rm junk.txt
4     This command deletes the file junk.txt.
5 Proceed? [y/N] y
6 $ rm junk.txt

```

Listing 7: Handling a destructive command with user confirmation.

```

1 $ doit "open vim to write a story"
2 doit: 'vim' opens an interactive program doit can't wait for. Run it
   yourself:
3     vim

```

Listing 8: Interactive program refusal.

3.4 Guard Override in Practice

The guard is not hypothetical. Across our 15 guard test cases it overrode the model's `is_destructive` flag **6 times** (`logs/phase2/safety_guard_results.json`). A common case is a redirect that writes a file, which the model easily under-flags as read-only:

```

1 command:          ls > files.txt
2 model is_destructive: false    (looks like a read-only listing)
3 guard is_destructive: true     (matched the ">" write-redirect pattern)
4 result:           Proceed? [y/N] gate shown before anything runs

```

Listing 9: A real guard override: the model reported the command as safe; the guard forced a confirmation.

3.5 Limitations

The regex guard trades precision for determinism, which produces both false positives and blind spots. A read-only command that merely *mentions* a destructive token — e.g. `grep "rm -rf" notes.txt` — is flagged destructive and gated behind a confirmation it does not need. Conversely, destruction hidden from a simple pattern match slips through: command substitution (`$(...)`), `xargs rm`, and similar indirection are not caught. Sudo and interactive-program detection is prefix-based, so a leading flag-with-value (e.g. `ssh -p 2222 host`) can be misread as non-interactive. These are accepted tradeoffs of a deterministic guard over a second LLM judge, not bugs; the model’s Layer-1 self-report is the complementary net.

3.6 ACDL Context Specification: DoitAgentSafety

The diagram below shows the only safety part the model actually sees: a `SAFETY_INSTRUCTIONS` block in the system prompt. The regex guard and the `Proceed? [y/N]` gate run in Python and send nothing to the model, so they are not drawn. In short, the diagram shows what the model is told about safety, not how we enforce it.



Figure 4: Visual representation of the ACDL context for DoitAgentSafety.

```
// doit v2---safety / confirm (Phase 2)
// Same single-command turn as v1, plus a SAFETY_INSTRUCTIONS block in the
// system prompt. The regex guard + y/N confirm are CONTROLLER logic (safety.py
// / _handle_run_command)---they send nothing to the model, so they are NOT in
// this spec. The only safety-related thing that reaches the context is the
// instructions block; as in v1 the command result is not fed back.

DoitAgentSafety[@T]: {
  S: {
    AGENT_INSTRUCTIONS
    SAFETY_INSTRUCTIONS           // destructive -> y/N gate; sudo/interactive
      blocked
    AVAILABLE_TOOLS(run_command, answer) // via API tools=
  }
  U: {
    ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
  }
  U: env.user_request[@T]
}
```

Listing 10: ACDL source for DoitAgentSafety (acd1/v2.safety.acd1).

4 Model flexibility

To ensure the utility can run independently of commercial API keys, we extended the system to run models locally as well.

4.1 Implementation Details

We introduced a configuration file `~/doit.cfg` to declare the active model and its adapter. We implement two code adapters behind a unified interface.

4.2 Design Decisions

Native vs. Prompted Adapter Architectures

Native Adapter (`_call_native`): For models supporting LiteLLM’s JSON tool-calling protocol (e.g., `gpt-4o-mini`, `qwen3:4b-instruct`). LiteLLM passes tool schemas out-of-band and receives structured tool outputs.

Prompted Adapter (`_call_prompted`): For local models that do not natively support tool-calling (e.g., `gemma3:4b`). It renders tool schemas as text, appends them to the system prompt, and enforces a strict JSON wrapper. To handle parser failures, we implement a string-based balanced brace scanner to locate JSON blocks, and allow **one retry** by appending the syntax error back to the LPU.

4.3 Evaluation of Local Models

We evaluated three models using the test suite in `tests/cases.md`:

Table 2: Model capabilities and adapters.

Model Name	Deployment	Adapter	Tool-Calling Capability
<code>openai/gpt-4o-mini</code>	API (Remote)	Native	Full structured tool support
<code>ollama/qwen3:4b-instruct</code>	Local (Ollama)	Native	Local tool-calling support
<code>ollama/gemma3:4b</code>	Local (Ollama)	Prompted	None; relies on text JSON prompts

We used the ~4B tier (`qwen3:4b-instruct`, `gemma3:4b`) that the assignment permits, rather than the 7–8B models our original plan named: we served the local models on our own machines, and downloading and running the larger models there was prohibitively slow.

4.4 Analysis of Divergences and Failure Modes

4.4.1 Gemma-3 Impossible Request Failure

When asked to perform a physically impossible command like `"make my laptop fly"`, the native models correctly responded with the conversational `answer` tool. However, the local `gemma3:4b` model (running under the prompted adapter) attempted to execute it via `bash`, choosing the `run_command` tool with:

```
1 echo "Sorry, I cannot make your laptop fly."
```

Because the model appended a trailing double quote, the command crashed with a shell parse error: `zsh:1: unmatched "`.

4.4.2 Prompted Adapter Recovery Logs

Under the prompted adapter, local models often return markdown fences and conversational preambles. Below is a log showing our parser successfully stripping the preamble and recovering the JSON payload:

```

1 Raw LPU Output:
2 Sure, here is the command to view disk space:
3 ```json
4 {
5   "tool": "run_command",
6   "args": {
7     "command": "df -h",
8     "is_destructive": false,
9     "explanation": "Check available disk space."
10  }
11 }
12 ```
13 Parser Action: Strip fences, run brace scan -> extracted {"tool": "
    run_command", ...}

```

Listing 11: JSON de-fencing and brace-scanning recovery.

4.5 Case-by-Case Results

We ran cases 1–8 (basic) and 56–60 (hard single-command discriminators) from `tests/cases.md` on all three models in single-command mode (`enable_plans=false`), so this is a clean translation test: one command per case, right or wrong. Table 3 grades each turn.

Table 3: Per-case results (V = correct command and outcome, ~ = partial, X = wrong / failed / incorrectly refused).

#	Case	gpt-4o-mini	qwen3:4b	gemma3:4b
1	hidden files (<code>ls -a</code>)	V	X	X
2	disk space (<code>df -h</code>)	V	V	V
3	impossible request → refuse	V	V	X
4	joke → refuse	V	V	V
5	“how do I” → answer	V	X	V
6	delete + confirm	V	V	V
7	delete + abort	V	~	V
8	redirect self-flag	V	V	V
56	5 largest, recursive, human-size	V	X	X
57	3 most common words	X	V	X
58	non-blank <code>.py</code> lines	X	X	X
59	files modified < 24h	V	X	X
60	rename <code>.txt</code> → <code>.md</code>	~	V	X
Correct count		10	7	6

The ranking is `gpt-4o-mini` (10/13) > `qwen3:4b` (7/13) > `gemma3:4b` (6/13). Case 58 ("count the total number of non-blank lines across all `.py` files here", expected 5) is a good discriminator — all three failed it. The clearest local-vs-local split is case 57 ("what are the 3 most common words in `notes.txt`?"): `qwen3` produced the correct pipeline (`the=4`, `apple=3`, `fox=2`), while `gemma3` mis-flagged the read-only pipeline as destructive and aborted.

`gpt-4o-mini` also failed this case, so case 57 is the one place a local model outperformed the API model. On case 59 ("list only the files modified in the last 24 hours, newest first") both local models emitted GNU-only `ls` flags (`--time-style`, `--block-size`) that fail on BSD/macOS, whereas `gpt-4o-mini` used a portable `find -mtime -1`.

4.5.1 Logged Reliability Numbers

- The deterministic regex guard overrode the model's own `is_destructive` flag in **6 of 15** guard test cases (`logs/phase2/safety_guard_results.json`); the same set also hard-blocked 1 `sudo` and 4 interactive commands.
- The prompted-adaptor JSON parser passed **10/10** offline cases (`logs/phase3/prompted_adapter_results`) covering fenced JSON, JSON-in-prose, braces-inside-strings, hallucinated-tool rejection, and the retry path.
- Live, `qwen3` (native) produced **no** JSON/tool-call failures across all 13 cases; `gemma3` (prompted) hit one malformed reply it recovered from and one hard failure after two retries.

4.6 Local-vs-Local Head-to-Head: `qwen3` vs `gemma3`

The assignment asks specifically for a comparison between a local model adapted for tool-calling (`qwen3:4b-instruct`, native) and one that is not (`gemma3:4b`, prompted JSON). They split the wins — neither is uniformly better — and the differences land exactly where structured output and tool-use decisions matter.

qwen wins (case 57, "what are the 3 most common words in notes.txt?"). `qwen` composes a correct pipeline and gets the right counts; `gemma` mis-flags the read-only pipeline as destructive and aborts, answering nothing:

```

1 # qwen3 (native) -- correct
2 $ grep -oE '([a-z]+)' notes.txt | sort | uniq -c | sort -nr | head -n 3
3     4 the
4     3 apple
5     2 fox
6 # gemma3 (prompted) -- over-flags a read-only pipeline, aborts, answers
   nothing
7 This command modifies the filesystem:
8     cat notes.txt > temp.txt && tr ... | sort | uniq -c | sort -nr |
   head -3
9 Proceed? [y/N] Aborted. (Nothing was executed.)

```

Listing 12: `qwen` produces the answer; `gemma` over-flags a read-only command and stops.

gemma wins (case 5, "how do I see hidden files?"). The tool-calling model is not always the smarter one: `gemma` correctly explains and runs nothing, while `qwen` wrongly executes instead of answering:

```

1 # gemma3 -- correct: explains, runs nothing
2 $ doit "how do I see hidden files?"
3 To view hidden files, use the 'ls -a' command ...
4 # qwen3 -- wrong: executes instead of answering
5 $ doit "how do I see hidden files?"
6 $ ls -a

```

Listing 13: `gemma` answers the question; `qwen` runs a command it should not.

Stability and structured output. On the prompted adaptor `gemma` is unstable run-to-run: the same joke request refused once and was told the next run, and case 60 ("rename every `.txt` file in this folder to have a `.md` extension instead") produced a malformed reply on

one run and a hard “no JSON after 2 attempts” failure on the next. qwen on the native adapter produced **zero** JSON/tool-call failures across all 13 cases — the native tool-calling protocol simply removes the JSON-parsing failure mode. qwen’s one quirk was the opposite of gemma’s recklessness: over-caution, refusing in case 1 (“show me all files here, including hidden ones”) to list a harmless `/tmp` sandbox at all.

History parroting. A further weakness shows over a longer session: once one command dominates gemma’s replayed history, it stops reading the request and just repeats that command. In one run it handled `delete junk.txt` and `save the listing to files.txt` correctly, then returned the same `ls > files.txt` for every following request — `install http`, `open vim`, even `forget the files` — copying its own last output instead of resolving the new instruction (logs/3A783293.jsonl). gpt-4o-mini handled the identical sequence correctly.

4.7 Limitations

Model flexibility is bounded by the weakest supported model. The 4B local models we tested (qwen3:4b-instruct, gemma3:4b) are materially less reliable than the API model (7/13 and 6/13 versus 10/13 in Table 3), and the prompted adapter grants only **one** JSON-repair retry before giving up — a persistently malformed reply aborts the turn. We also observed model-specific quirks the framework cannot fix from outside: `gemma3` mis-flagging read-only pipelines as destructive, and both local models emitting GNU-only flags that fail on BSD/macOS. The adapter guarantees a *uniform interface*, not uniform competence.

4.8 ACDL Context Specification: DoitAgentPrompted

For models without native tool-calling, tool schemas move into the system prompt prose, and the raw text reply is parsed. The visual ACDL diagram below shows how we handle JSON validation and retries:

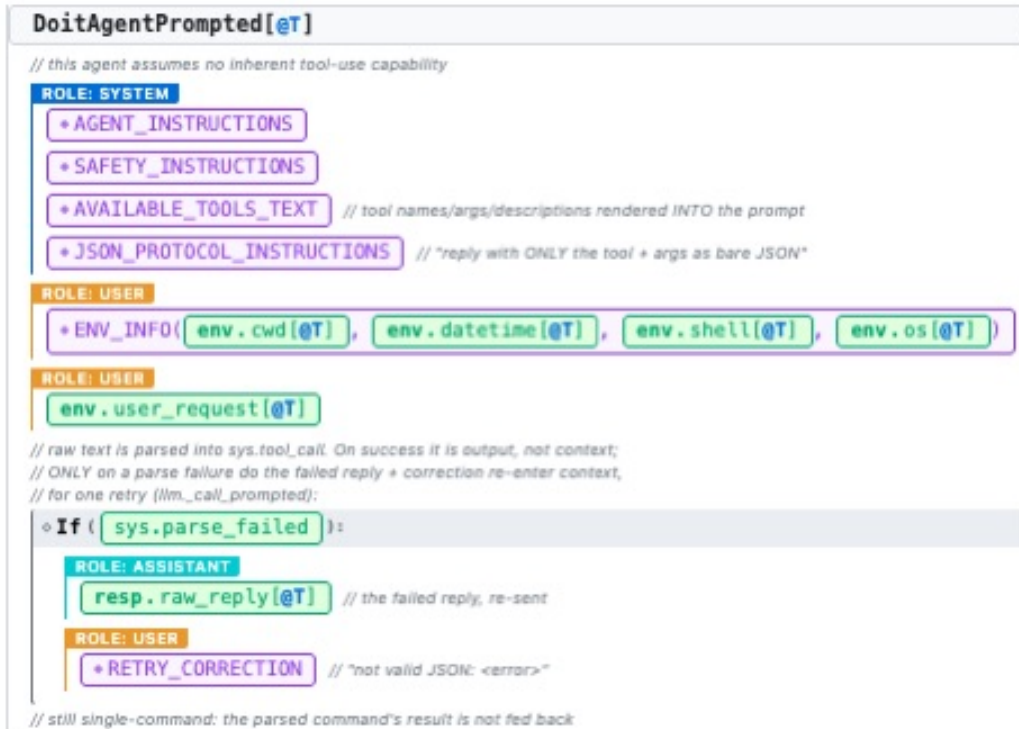


Figure 5: Visual representation of the ACDL context for DoitAgentPrompted.


```
// doit v3---the PROMPTED adapter (Phase 3, model flexibility)
// Same agent / tools / safety as v2---only the plumbing to the LPU differs, for
// local models WITHOUT native tool-calling. Two changes are visible here:
// 1. the tools move INTO the prompt as text (AVAILABLE_TOOLS_TEXT +
//    JSON_PROTOCOL_INSTRUCTIONS), instead of the native out-of-band tools=.
// 2. the decision arrives as raw model text we parse. The one-shot retry IS
//    context (unlike the safety gate) because RETRY_CORRECTION is itself a
//    real U: message sent back to the model.
// This resp.raw_reply -> sys.tool_call parse is the deliberate diff vs v2.

DoitAgentPrompted[@T]: {
  // this agent assumes no inherent tool-use capability
  S: {
    AGENT_INSTRUCTIONS
    SAFETY_INSTRUCTIONS
    AVAILABLE_TOOLS_TEXT                                // tool names/args/descriptions
      rendered INTO the prompt
    JSON_PROTOCOL_INSTRUCTIONS                          // "reply with ONLY the tool + args
      as bare JSON"
  }
  U: {
    ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
  }
  U: env.user_request[@T]

  // raw text is parsed into sys.tool_call. On success it is output, not context;
  // ONLY on a parse failure do the failed reply + correction re-enter context,
  // for one retry (llm._call_prompted):
  If sys.parse_failed {
    A: resp.raw_reply[@T]                                // the failed reply, re-sent
    U: RETRY_CORRECTION                                  // "not valid JSON: <error>"
  }
  // still single-command: the parsed command's result is not fed back
}
```

Listing 14: ACDL source for DoitAgentPrompted (acd1/v3.prompted.acd1).

5 Multi-turn

To move from isolated commands to conversational sequences, we carry state across invocations, allowing commands to refer to previous outputs.

5.1 Implementation Details

When `doit` runs, it loads the session ID from the environment variable `DOIT_SESSION` (which defaults to "default" if unset). The Controller reads and parses the JSONL session logs located at `~/.doit/sessions/$DOIT_SESSION.jsonl`, extracts the last $K = 10$ turns, and splices them back into the prompt context.

5.2 Design Decisions

JSONL Storage Format

Choice: We chose JSONL (JSON Lines) files over SQLite. JSONL is lightweight, human-readable, and allows $O(1)$ appends. It facilitates debugging and log evidence curation for the report.

Tiered Truncation Budgets (HOT vs. COLD)

Problem: Feeding complete command outputs (e.g. file listings or compiler logs) back into the prompt quickly overflows the context window.

Solution: We implement a tiered head/tail truncation strategy. The active turn's output is allocated a **Hot Budget** (up to 3,000 characters head, 1,000 characters tail) to support follow-up decisions. Older replayed history turns are truncated to a **Cold Budget** (1,000 characters head, 300 characters tail), which provides enough context to resolve follow-ups (like referring to listed files) while saving tokens.

5.3 Example Multi-Turn Interactions

Below is a live multi-turn session log using `gpt-4o-mini`, demonstrating how the model changes flags and behavior rather than parroting commands:

```

1 $ DOIT_SESSION=demo doit "list the files here"
2 $ ls -l
3 total 232
4 -rw-r--r--@  1 yuval  staff  12405 Jul  7 18:24 notes.txt
5 -rw-r--r--@  1 yuval  staff  30675 Jul  7 18:57 report.txt
6
7 $ DOIT_SESSION=demo doit "now sort them by date"
8 $ ls -lt
9 total 232
10 -rw-r--r--@  1 yuval  staff  30675 Jul  7 18:57 report.txt
11 -rw-r--r--@  1 yuval  staff  12405 Jul  7 18:24 notes.txt
12
13 $ DOIT_SESSION=demo doit "now in ascending order"
14 $ ls -ltr
15 total 232
16 -rw-r--r--@  1 yuval  staff  12405 Jul  7 18:24 notes.txt
17 -rw-r--r--@  1 yuval  staff  30675 Jul  7 18:57 report.txt
18
19 $ DOIT_SESSION=demo doit "by creation time"
20 The 'ls' command on macOS does not support sorting by creation time
21 directly; it can only sort by modification time. Would you like to sort
22 by modification time instead?
```

Listing 15: Relational multi-turn listing and sorting.

The final turn shows the agent being honest about a platform limitation rather than parroting a flag: macOS/BSD `ls` has no portable creation-time sort, so it answers instead of emitting a command that would not do what was asked (`logs/phase4/live_multiturn_gpt4omini.txt`).

5.4 Limitations

History is a fixed window of the last $K = 10$ turns; anything older is dropped entirely rather than summarized, so a reference to a command from twelve turns ago cannot be resolved. Compaction (folding evicted turns into a running digest) is the natural remedy but is described, not implemented. Separately, when the shell snippet is not installed `DOIT_SESSION` falls back to "default", and all terminals then share a single history stream.

5.5 ACDL Context Specification: DoitAgentHistory

The ACDL diagram below adds the replayed conversation history to the context:

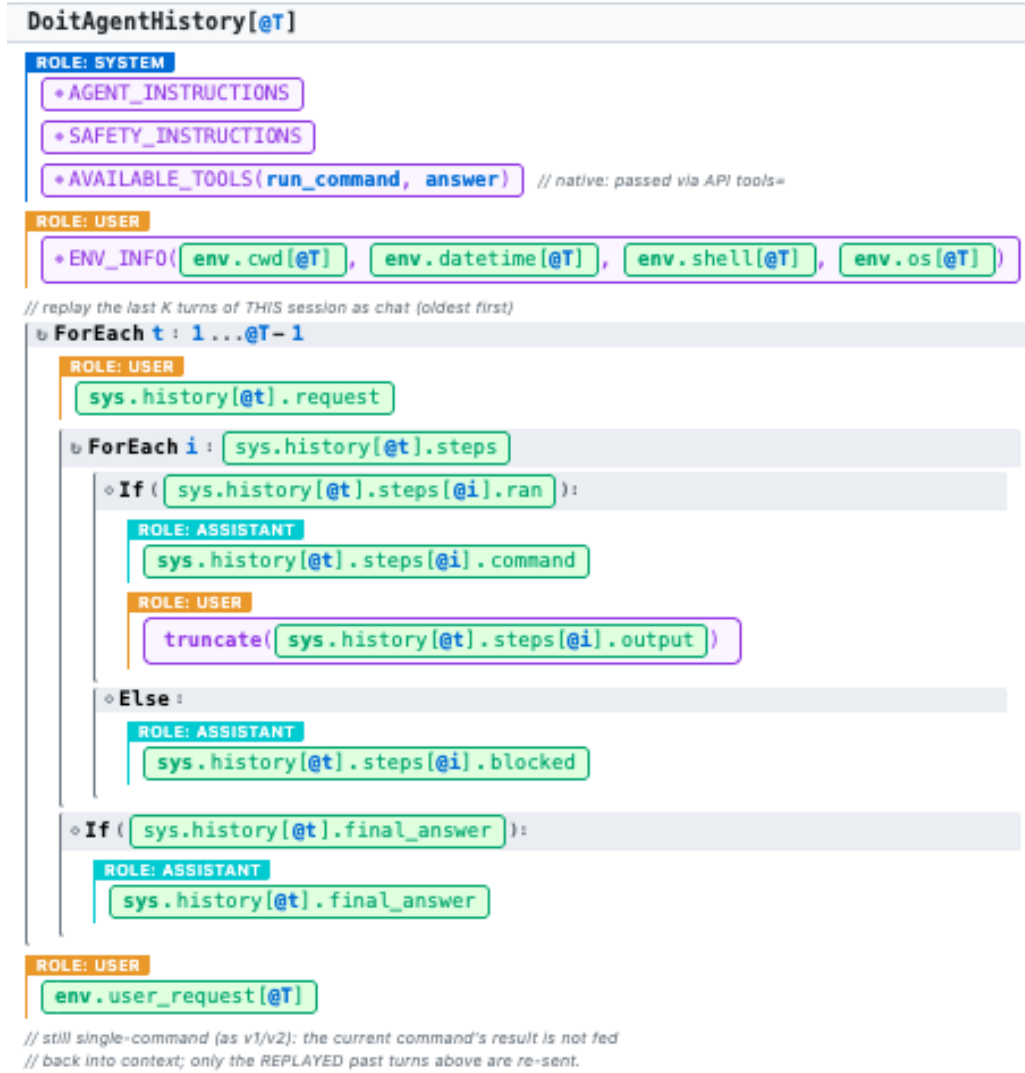


Figure 6: Visual representation of the ACDL context for DoitAgentHistory.

```

// doit v4---multi-turn history (Phase 4)
// Same single decision + safety as v2. New: replay THIS session's last K turns
// as chat BEFORE the current request, so a follow-up ("now sort them by date")
// resolves against the previous turn. Older turns use the COLD truncation
// budget, the live turn HOT (tools.truncate_for_context)---a controller detail,
// shown here only as truncate(...). History is an action history -> sys.history.

DoitAgentHistory[@T]: {
  S: {
    AGENT_INSTRUCTIONS
    SAFETY_INSTRUCTIONS
    AVAILABLE_TOOLS(run_command, answer) // native: passed via API tools=
  }
  U: {
    ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
  }

  // replay the last K turns of THIS session as chat (oldest first)
  ForEach(@t: range(1, @T-1)) {
    U: sys.history[@t].request
    ForEach(@i: sys.history[@t].steps) {
      If sys.history[@t].steps[@i].ran {
        A: sys.history[@t].steps[@i].command
        U: truncate(sys.history[@t].steps[@i].output)
      }
    }
    Else {

```

```

    A: sys.history[@t].steps[@i].blocked
  }
}
If sys.history[@t].final_answer {
  A: sys.history[@t].final_answer
}
}

U: env.user_request[@T]
// still single-command (as v1/v2): the current command's result is not fed
// back into context; only the REPLAYED past turns above are re-sent.
}

```

Listing 16: ACDL source for DoitAgentHistory (acd1/v4_history.acdl).

6 Clarifications

When a user request is ambiguous, the agent must ask clarifying questions rather than guessing or executing a potentially harmful action.

6.1 Implementation Details

We introduce the `ask_user` tool. When invoked by the LPU, the Controller prints the question alongside a numbered options menu, blocks on user input, and feeds the response back to the LPU. This process runs as a within-turn loop and does not count towards the command-step count.

6.2 Design Decisions

Unanswered Clarification Default Behavior

Problem: How to proceed if the user presses Enter (empty input) or interrupts the prompt.

Solution: Pressing Enter returns "no answer" to the LPU, instructing it to proceed with the default option it stated. If this default results in a destructive command, the Layer 2 safety confirmation gate (Proceed? [y/N]) still blocks execution. Pressing Ctrl-C raises a KeyboardInterrupt, aborts the turn immediately, and logs the abort status.

6.3 Non-Annoyance Cap

To prevent the agent from asking questions repeatedly, we implement two controls:

1. **System Prompt Policy:** The LPU is instructed to ask questions only when a wrong guess is destructive, or when interpretations lead to different results with no clear default. Otherwise, it must assume the most common interpretation and state it in the final output.
2. **Structural Cap:** We enforce a hard limit of `MAX_CLARIFICATIONS=2` in the Controller. If the LPU exceeds this, the Controller feeds a capped message warning the model to commit to its best guess.

6.4 Example Interaction

Below is a live interaction showing how the clarification loop and safety confirmation gate stack together cleanly:

```
1 $ doit "delete the logs"
2 Are you sure you want to delete the logs? This action cannot be undone.
3   1. Yes
4   2. No
5 Your answer (number or text): 1
6
7 [WARNING] This command modifies the filesystem:
8   rm *.log
9   This command deletes log files in the current directory.
10 Proceed? [y/N] y
11 $ rm *.log
```

Listing 17: Stacked safety: clarification leading to a destructive confirmation gate.

6.5 Limitations

The empty-input default (Enter → "no answer" → proceed with the stated default) is safe for destructive commands, which the Proceed? [y/N] gate still catches, but a *read-only* default can still surprise a user who expected the question to block. The Ctrl-C abort path is logged but has not been stress-tested. Finally, the MAX_CLARIFICATIONS=2 cap forces the model to commit to a best guess after two questions, trading a possible wrong interpretation for guaranteed non-annoyance.

6.6 ACDL Context Specification: DoitAgentClarify

The final formal context structure for our agent, representing both replayed turns and the within-turn clarification loop, is detailed in the visual ACDL diagram below:

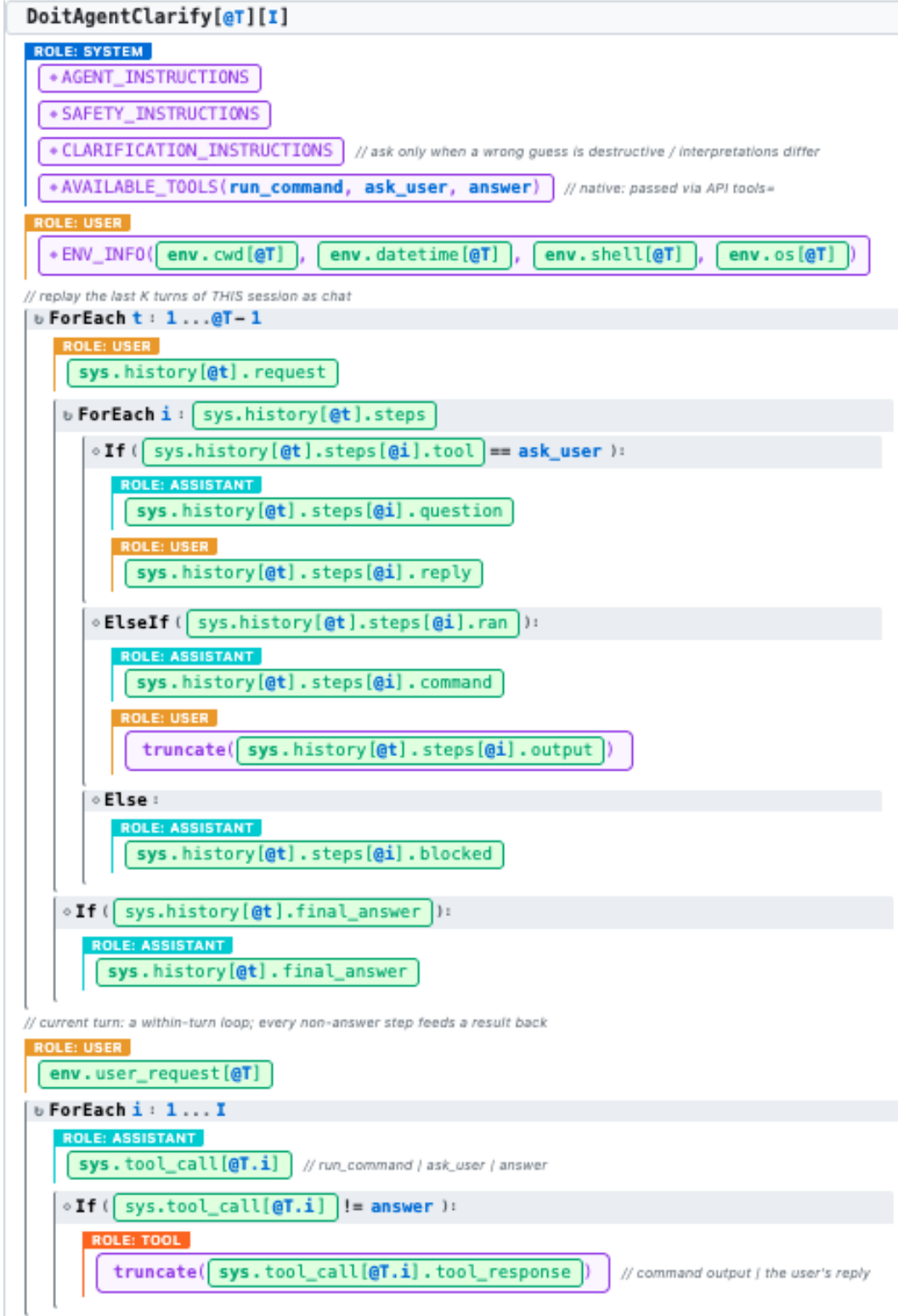


Figure 7: Visual representation of the ACDL context for DoitAgentClarify (clarifications and multi-turn history).

```

// doit v5--- clarifications + richer interactions (Phase 5)
// Builds on v4. New: ask_user(question, options?) and a WITHIN-TURN loop---
// ask_user does NOT end the turn; the reply is fed back and the model is
// re-called, so @T.I now ranges over several steps in one turn. THIS is where a
// tool result first re-enters context (v1-v4 never fed one back). The

```

```

// clarification cap (MAX_CLARIFICATIONS) and empty-input default are controller
// logic; the model just sees back the one reply message as the tool_response.

DoitAgentClarify[@T, I]: {
  S: {
    AGENT_INSTRUCTIONS
    SAFETY_INSTRUCTIONS
    CLARIFICATION_INSTRUCTIONS // ask only when a wrong guess is
      destructive / interpretations differ
    AVAILABLE_TOOLS(run_command, ask_user, answer) // native: passed via API tools=
  }
  U: {
    ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
  }

  // replay the last K turns of THIS session as chat
  ForEach(@t: range(1, @T-1)) {
    U: sys.history[@t].request
    ForEach(@i: sys.history[@t].steps) {
      If sys.history[@t].steps[@i].tool == ask_user {
        A: sys.history[@t].steps[@i].question
        U: sys.history[@t].steps[@i].reply
      } ElseIf sys.history[@t].steps[@i].ran {
        A: sys.history[@t].steps[@i].command
        U: truncate(sys.history[@t].steps[@i].output)
      } Else {
        A: sys.history[@t].steps[@i].blocked
      }
    }
    If sys.history[@t].final_answer {
      A: sys.history[@t].final_answer
    }
  }

  // current turn: a within-turn loop; every non-answer step feeds a result back
  U: env.user_request[@T]
  ForEach(@i: range(1, I)) {
    A: sys.tool_call[@T.i] // run_command | ask_user | answer
    If sys.tool_call[@T.i] != answer {
      T: truncate(sys.tool_call[@T.i].tool_response) // command output | the user's
        reply
    }
  }
}

```

Listing 18: ACDL source for DoitAgentClarify (acd1/v5.clarify.acd1).

7 Richer interactions

Richer interactions require the agent to respond to non-commands (e.g., “how do I do X?”) and handle follow-up refinement chains (e.g., “how do I list files?” → “modify it to show sizes” → “execute it”).

7.1 Implementation Details

We resolved this without introducing new tools or classification models.

- **Explaining vs. Running:** If a user asks a question, the LPU uses the `answer` tool. The Controller displays the answer and terminates the turn loop without executing commands.
- **Execution from History:** When the user subsequently requests “execute it”, the request is sent with the session’s turn history. The LPU reads the command structure it described in the previous `answer` and passes it to the `run_command` tool.

7.2 Example Interaction

Below is a three-turn interaction chain showing history extraction and execution:

```

1 $ doit "how do I find python files modified this week?"
2 You can use: find . -name "*.py" -mtime -7
3 $ doit "modify it to also show file sizes"
4 You can use: find . -name "*.py" -mtime -7 -ls
5 $ doit "execute it"
6 $ find . -name "*.py" -mtime -7 -ls
7 14283      8 -rw-r--r--    1 yuval  staff   1319 Jul  7 15:00 ./train.py

```

Listing 19: How-do-I question, command modification, and execution.

7.3 What Worked and What Didn’t: the Repeated-Question Bug

An earlier version had a subtle bug: a past turn changed how a later, identically-worded turn was handled. Turn 1 (a literal question) was correctly answered; turn 2 (a modification) correctly ran a command; but turn 3 — word-for-word the same as turn 1 — ran the command instead of answering.

```

1 $ doit "how do i find python files modified this week?"
2 You can find Python files ... find . -name "*.py" -mtime -7    <- turn
   1: answered (correct)
3 $ doit "modify it to also show file sizes"
4 $ find . -name "*.py" -mtime -7 -exec ls -lh {} \;             <- turn 2:
   ran (correct)
5 $ doit "how do i find python files modified this week?"
6 $ find . -name "*.py" -mtime -7 -exec ls -lh {} \;             <- turn 3:
   RAN, should have answered

```

Listing 20: The bug: an identical question runs the second time, because a command turn came in between (logs/phase5/live_richer_interactions_history_bleed.txt).

The fix that did not work. We first added a new, separate rule to the system prompt: a request phrased as a question always gets **answer**. It made no difference — turn 3 still ran the command.

Why. The prompt already had an action-verb rule, placed earlier, that listed **find** by name (“if the request describes a concrete action — create, ..., find, ... — run it”). Our new exception sat in its own paragraph further down. The two rules never met on the page, so faced with “how do i *find* ...” the model matched the concrete word **find** in the earlier, more specific rule and ran the command. A general exception in a distant paragraph could not override a specific keyword match.

The fix that worked. We moved the exception *inside* the action-verb rule, at the exact point of conflict: “...run it. EXCEPTION: a request literally phrased as a question is always **answer** — the question wording overrides the action verb.” Turn 3 then answered correctly.

Lesson. Prompt rules conflict by proximity and specificity, not just by being present: a general exception must live *inside* the specific rule it is meant to override, not in a separate paragraph. The same lesson recurred in Section 9, where a same-turn warning only changed the model’s behavior once it was moved into the tool-result text at the point of decision.

7.4 Limitations

We observed that smaller local models (such as `gemma3:4b`) frequently fail on the third turn (“execute it”), responding with “Execute what?” or hallucinating unrelated commands. This indicates that tracking command references across multiple conversational turns is a capability limited to larger, instruction-tuned models.

7.5 ACDL Context

Richer interactions add no new context block: answering “how do I ...” is a plain **answer**, and resolving a later “execute it” relies entirely on the multi-turn history replay. The relevant ACDL is therefore the history specification already shown in Figure 6 (Section 5).

8 Memory

The memory subsystem allows the agent to persist long-term user facts across terminal windows, directories, and session boundaries.

8.1 Implementation Details

Memory storage is managed in `doitlib/state.py` via the flat JSON file `~/.doit/memories.json` (using the functions `load_memories`, `add_memory`, and `forget_memory`). Unlike session history, this file is global and shared across all sessions.

During context assembly, the controller invokes `memory_block()` in `doitlib/context.py`, which formats the current memories using the template in `prompts/memory_block.txt`. This block lists each stored fact with a short stable id (e.g., `m1`, `m2`) and injects them as `sys.memories`. This goes in its own **user** message, not the system prompt: only `agent_instructions()` uses the **system** role, and every other block (memory included) is a **user** message.

The LPU manipulates these facts via two specialized tools: `remember(fact)` and `forget(id)`. Within the controller’s turn loop, memory modifications are treated as within-turn steps: when the agent issues a memory tool call, the controller executes the state change in `_handle_memory` and immediately re-enters the execution loop without decrementing the command step limit (the rationale is in the design decision below).

8.2 Design Decisions

Global Shared Storage vs. Session-Keyed Memory

Alternative Rejected: Storing memories inside the terminal’s isolated session history.

Rationale: Storing facts per session defeats the purpose of memory. If a user defines their project directory in one terminal, they expect that preference to be recognized when starting a fresh terminal or working in a different directory. Storing memories globally ensures they are visible across all terminal sessions.

Memory Operations as Within-Turn Steps

Problem: Ensuring that metadata operations (like saving a fact) do not block or consume the execution budget.

Rationale: If a user issues a command like “`remember this is my code folder and list files in it`”, the model must call `remember` followed by `run_command`. If memory tool calls counted towards the command step budget (which defaults to 1), the agent would stop after memorizing, failing to execute the command. Processing memory tools as within-turn steps resolves this constraint.

8.3 Example Interaction

Below is a multi-session interaction where a memory saved in one terminal window is retrieved in another:

```

1 Terminal A:
2 $ doit "remember that ~/school/llms/ass3 is my LLM class project folder"
3 Fact stored.
4
5 Terminal B:
6 $ doit "go to my LLM class project folder"
7 $ change_dir ~/school/llms/ass3

```

Listing 21: Memory storage and cross-session retrieval.

8.4 What Worked and What Didn't: a Memory Edit that Crashed

Editing a memory (“I changed my mind — ask me each time instead”, meant to drop one preference) crashed, and each retry deleted another memory:

```

1 memories before: [m1 = "project folder", m2 = "prefers ls sorted by
  size"]
2 $ doit "I changed my mind -- ask me each time instead"
3 forgot m2
4 doit: error: OpenAIException 400 -- 'tool_calls' must each be followed
  by a tool
5 response (one was left unanswered)
6 $ doit "I changed my mind -- ask me each time instead"      <- retry
7 forgot m1                                                    <- forgets
  the WRONG memory
8 memories after: []      (both facts gone)

```

Listing 22: The bug: a memory edit crashes and quietly deletes memories (logs/phase6/behavior.issue.txt).

Why. gpt-4o-mini put `forget` and `remember` in one message as two tool-calls. Our controller handles one decision per call, so it acted on the first and replayed the message with the second call still unanswered — which OpenAI rejects with a 400. Because `forget` had already run before the crash, every retry deleted one more memory.

The fix. Before replaying a message we now drop the extra tool-calls (`llm._dump_single_tool_call`); the dropped action simply runs on the next loop step, after the model sees the updated state. The edit is then crash-free:

```

1 $ doit "my favourite folder was changed to /prompts"
2 forgot m2
3 remembered [m2]: the user's favourite folder is /prompts
4 (m1 kept, m2 updated -- no crash, no wrong deletion)

```

Listing 23: After the fix: the edit updates one memory and keeps the other (logs/phase6/good.interaction.memo.txt).

8.5 Limitations

Limitation: Monotonic ID Generation and Reuse

Behavior: Memory IDs are generated dynamically as $m\{\text{max_id} + 1\}$. If the highest ID is forgotten, its identifier is recycled for the next memory addition.

Limitation: While this keeps identifiers short and readable, it creates a potential reference conflict in long-term history records. If an old replayed turn references a recycled ID, the context might contain a different fact than what the model originally saw. We accepted this limitation due to the small scale of memories (typically under a dozen facts).

8.6 ACDL Context Specification: DoitAgentMemory

The ACDL diagram below adds the persistent-memory block to the context:

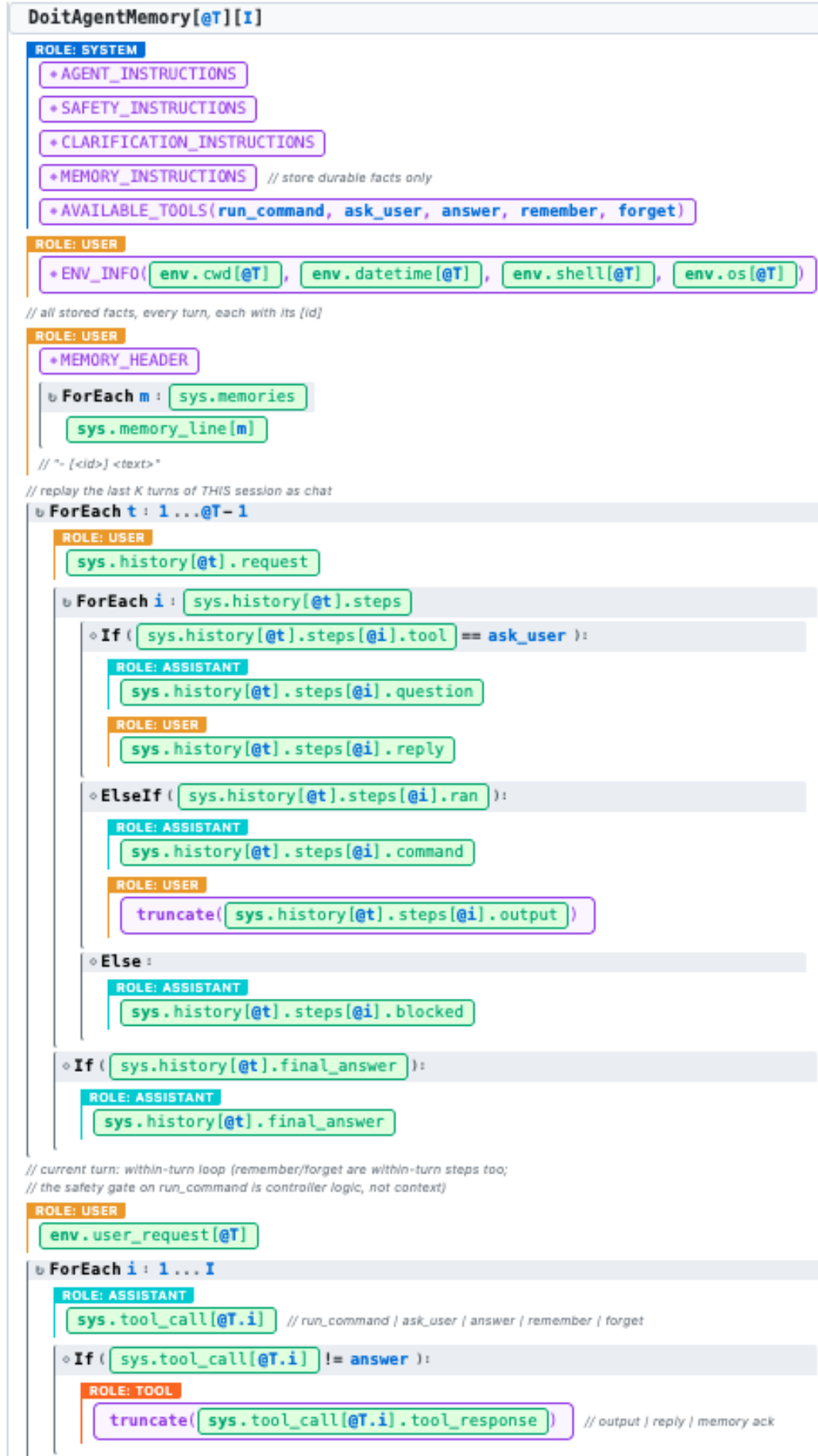


Figure 8: Visual representation of the ACDL context for memory injection.

```

// doit v6---persistent memory (Phase 6)
// Builds on v5. New tools remember(fact)/forget(id) and a new context block:
// sys.memories---ALL stored facts, injected every turn, each with its visible
// [id] (load-bearing: it's how the model names which fact to forget). Not
// session-keyed---a shared store, so it sits with the ambient S/U setup, above
// the per-session history loop. A memory op is a within-turn step; its ack is
// just that step's tool_response.

DoitAgentMemory[@T, I]: {
  S: {
    AGENT_INSTRUCTIONS
    SAFETY_INSTRUCTIONS
    CLARIFICATION_INSTRUCTIONS
    MEMORY_INSTRUCTIONS // store durable facts only
    AVAILABLE_TOOLS(run_command, ask_user, answer, remember, forget)
  }
  U: {
    ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
  }
  // all stored facts, every turn, each with its [id]
  U: {
    MEMORY_HEADER
    ForEach(m: sys.memories) { sys.memory_line[m] } // "- [id] <text>"
  }
  // replay the last K turns of THIS session as chat
  ForEach(@t: range(1, @T-1)) {
    U: sys.history[@t].request
    ForEach(@i: sys.history[@t].steps) {
      If sys.history[@t].steps[@i].tool == ask_user {
        A: sys.history[@t].steps[@i].question
        U: sys.history[@t].steps[@i].reply
      } ElseIf sys.history[@t].steps[@i].ran {
        A: sys.history[@t].steps[@i].command
        U: truncate(sys.history[@t].steps[@i].output)
      } Else {
        A: sys.history[@t].steps[@i].blocked
      }
    }
    If sys.history[@t].final_answer {
      A: sys.history[@t].final_answer
    }
  }
  // current turn: within-turn loop (remember/forget are within-turn steps too;
  // the safety gate on run_command is controller logic, not context)
  U: env.user_request[@T]
  ForEach(@i: range(1, I)) {
    A: sys.tool_call[@T.i] // run_command | ask_user | answer
    | remember | forget
    If sys.tool_call[@T.i] != answer {
      T: truncate(sys.tool_call[@T.i].tool_response) // output | reply | memory ack
    }
  }
}
}

```

Listing 24: ACDL source for DoitAgentMemory (acd1/v6.memory.acdl).

9 Changing directories (change_dir)

Before adding user awareness, we fixed the `cd` trap (see the wrapper decision in Section 2). A Python subprocess cannot change its parent shell's directory, so we added a `change_dir(path)` tool. It checks that the path exists and writes it to `~/.doit/cd_target_${DOIT_SESSION}`; the shell wrapper reads that file after `doit` exits and runs the real `cd`. Like `remember/forget`, it is a within-turn step, so “go there and list it” can call `change_dir` and then `run_command` in the same turn.

One catch we found while testing: a `run_command` in the same turn still runs in the *old* directory, because the real `cd` only happens after the process exits. So the model has to point a

same-turn command at the resolved path. The path check and the file write are Controller/shell work, so the diagram only shows the one result string the model gets back.

Limitations. `change_dir` depends entirely on the installed shell wrapper: if the `doit()` function is not present in the user's `~/.zshrc`/`~/.bashrc`, the tool validates the path and writes the target file, but the parent shell never reads it, so the `cd` silently no-ops (the same wrapper-dependence noted in Section 5). The path is checked at decision time, so a directory removed between that check and the wrapper's `cd` would also fail silently in the shell.



Figure 9: Visual representation of the ACDL context for DoitAgentChangeDir.

```

// doit v6.5--- change_dir (the cd-trap fix, inserted before Phase 7)
// Builds on v6. New tool change_dir(path), a within-turn step. The cd trap: a
// subprocess can't change its parent shell's cwd, so change_dir writes the

```

```

// target to ~/.doit/cd_target_${DOIT_SESSION} and the doit() shell wrapper reads
// it after exit and performs the real cd---CONTROLLER / shell logic, not
// context, so it is not shown. The model only sees back one result: "will cd
// after exit (qualify a same-turn command)" or "no such directory".

DoitAgentChangeDir[@T, I]: {
  S: {
    AGENT_INSTRUCTIONS
    SAFETY_INSTRUCTIONS
    CLARIFICATION_INSTRUCTIONS
    MEMORY_INSTRUCTIONS
    CHANGE_DIR_INSTRUCTIONS // change_dir vs cd; same-turn
    command targets resolved path
    AVAILABLE_TOOLS(run_command, ask_user, answer, remember, forget, change_dir)
  }
  U: {
    ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
  }

  U: {
    MEMORY_HEADER
    ForEach(m: sys.memories) { sys.memory_line[m] }
  }

  // replay the last K turns of THIS session as chat
  ForEach(@t: range(1, @T-1)) {
    U: sys.history[@t].request
    ForEach(@i: sys.history[@t].steps) {
      If sys.history[@t].steps[@i].tool == ask_user {
        A: sys.history[@t].steps[@i].question
        U: sys.history[@t].steps[@i].reply
      } ElseIf sys.history[@t].steps[@i].ran {
        A: sys.history[@t].steps[@i].command
        U: truncate(sys.history[@t].steps[@i].output)
      } Else {
        A: sys.history[@t].steps[@i].blocked
      }
    }
    If sys.history[@t].final_answer {
      A: sys.history[@t].final_answer
    }
  }

  // change_dir is a within-turn step; composes with run_command in one turn.
  // Path validation + cd_target write are controller work (not shown).
  U: env.user_request[@T]
  ForEach(@i: range(1, I)) {
    A: sys.tool_call[@T.i] // ... | change_dir
    If sys.tool_call[@T.i] != answer {
      T: truncate(sys.tool_call[@T.i].tool_response) // change_dir -> "will cd after
        exit" | "no such directory"
    }
  }
}

```

Listing 25: ACDL source for DoitAgentChangeDir (acd1/v6.5_changedir.acd1).

10 User awareness

To enhance agent comprehension of manual shell actions, we capture the user’s manual terminal command history and inject it into the agent’s context.

10.1 Implementation Details

Shell integration snippets (appended to ~/.zshrc or ~/.bashrc) define a shell hook (precmd in Zsh, PROMPT_COMMAND in Bash) that automatically writes every shell command executed by the user to a per-session log file: ~/.doit/shell_hist/\${DOIT_SESSION} in the format timestamp|cwd|command.

When `doit` runs, the controller calls `load_recent_user_commands` in `doitlib/state.py`. This helper reads the raw shell history and filters out commands starting with the word `doit` to isolate manual user actions. The controller then compiles the last 20 filtered commands using `user_shell_history_block` in `doitlib/context.py` (following the template in `prompts/user_shell_history_block.txt`) and appends them to the context as a list of manual command entries.

10.2 Design Decisions

Python-Side Log Filtering vs. Shell Hook Filtering

Alternative Rejected: Filtering out `doit` invocations inside the shell configuration hook itself.

Rationale: Shell-side configurations are fragile and can easily crash prompt drawing or command resolution if they hit syntax errors. We chose to log all commands indiscriminately from the shell, delegating the filtering and parsing logic to Python where it can be managed safely.

Isolated Per-Terminal History Streams

Rationale: Logging command history under a session-keyed folder `shell_hist/$DOIT.SESSION` guarantees that manual actions typed in Terminal A do not pollute Terminal B's context, maintaining local alignment.

10.3 Example Interaction

Below is a live session where the agent retrieves and explains a manual command failure:

```
1 $ mkdir reports
2 $ cd reports
3 $ cat status.log
4 cat: status.log: No such file or directory
5 $ doit "why did my last command fail?"
6 The command 'cat status.log' failed because there is no file named '
  status.log' inside the 'reports' folder you just created.
```

Listing 26: Retrieving manual history to explain command failure.

The assignment's headline case, "summarize what I just did", additionally shows the user/`doit` split: the summary is built only from the commands the user typed, while `doit`'s own invocations are filtered out (by `load_recent_user_commands`) before the block ever reaches the model (`logs/phase7/live_summarize_gpt4omini.txt`):

```
1 $ mkdir data2
2 $ cd logs
3 $ cd ..
4 $ mkdir data3
5 $ doit "summarize what I just did"
6 You created two directories named data2 and data3 in your current
  project
7 folder, and navigated into the logs directory and back. (The earlier
8 doit "... " calls are excluded -- they are the agent's own actions, not
  yours.)
```

Listing 27: Summarizing manual actions; prior `doit` invocations are excluded, not counted as user activity.

10.4 Limitations

Limitation: Manual Dotfile Synchronization

Problem: Updates to the snippet template in `shell/zshrc_snippet.sh` do not automatically synchronize with the user's active configuration file.

Limitation: If the shell hooks change in the codebase, the user must manually re-copy and re-source the snippet. Automating this sync was rejected to avoid corrupting dotfiles.

10.5 ACDL Context Specification: `DoitAgentUserAware`

The ACDL diagram below adds the manual user shell-history block to the context:

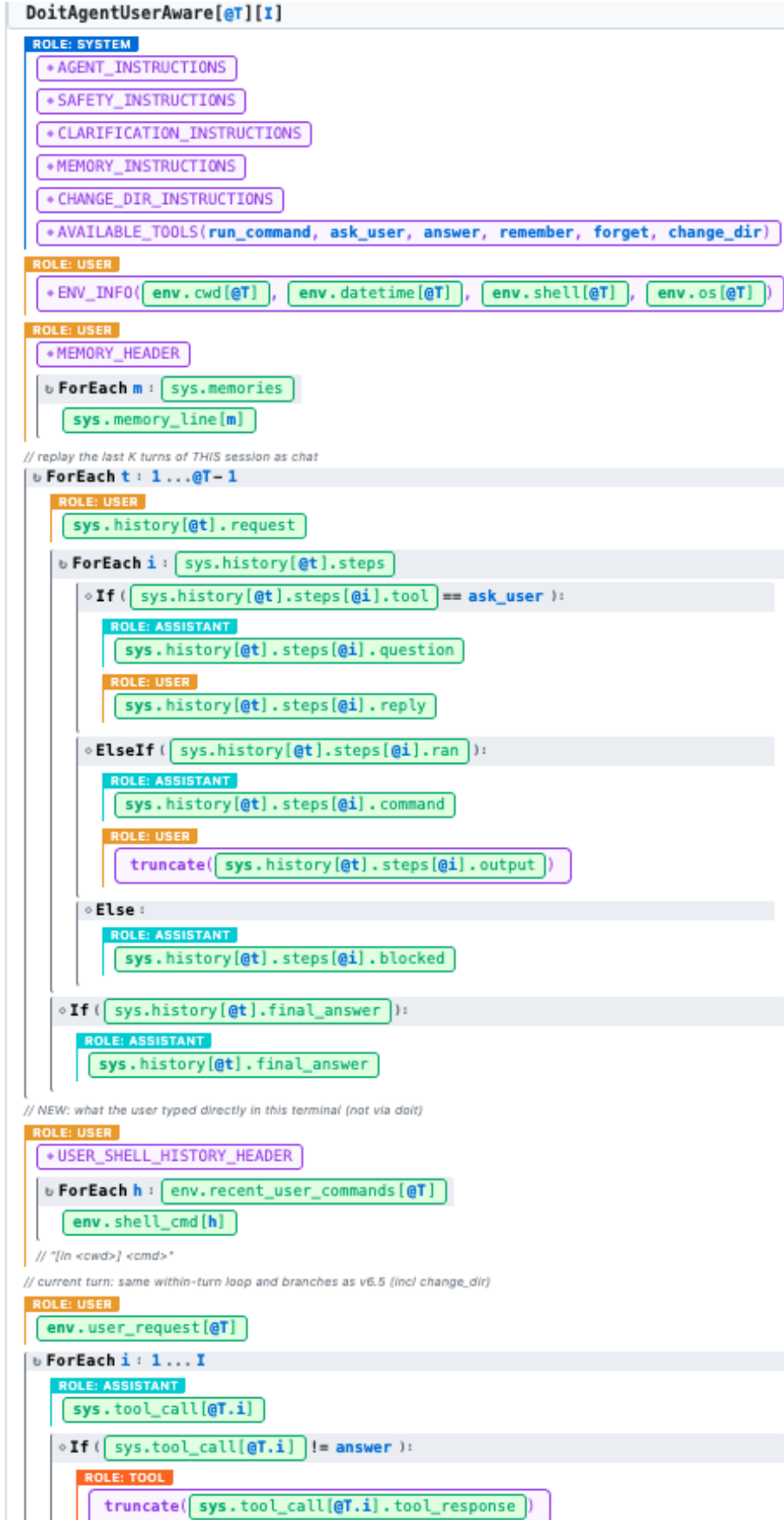


Figure 10: Visual representation of the ACDL context for user shell history awareness.

```

// doit v7---user shell-history awareness (Phase 7)
// Builds on v6.5. No new tool---a new context SOURCE: USER_SHELL_HISTORY, the
// commands the user typed directly in this terminal (a shell hook logs them to
// shell_hist; doit's own invocations are filtered out, block skipped when
// empty). Spliced after the replayed history, before the current request.

DoitAgentUserAware[@T, I]: {
  S: {
    AGENT_INSTRUCTIONS
    SAFETY_INSTRUCTIONS
    CLARIFICATION_INSTRUCTIONS
    MEMORY_INSTRUCTIONS
    CHANGE_DIR_INSTRUCTIONS
    AVAILABLE_TOOLS(run_command, ask_user, answer, remember, forget, change_dir)
  }
  U: {
    ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
  }

  U: {
    MEMORY_HEADER
    ForEach(m: sys.memories) { sys.memory_line[m] }
  }

  // replay the last K turns of THIS session as chat
  ForEach(@t: range(1, @T-1)) {
    U: sys.history[@t].request
    ForEach(@i: sys.history[@t].steps) {
      If sys.history[@t].steps[@i].tool == ask_user {
        A: sys.history[@t].steps[@i].question
        U: sys.history[@t].steps[@i].reply
      } ElseIf sys.history[@t].steps[@i].ran {
        A: sys.history[@t].steps[@i].command
        U: truncate(sys.history[@t].steps[@i].output)
      } Else {
        A: sys.history[@t].steps[@i].blocked
      }
    }
    If sys.history[@t].final_answer {
      A: sys.history[@t].final_answer
    }
  }

  // NEW: what the user typed directly in this terminal (not via doit)
  U: {
    USER_SHELL_HISTORY_HEADER
    ForEach(h: env.recent_user_commands[@T]) { env.shell_cmd[h] } // "[in <cwd>] <cmd>"
  }

  // current turn: same within-turn loop and branches as v6.5 (incl change_dir)
  U: env.user_request[@T]
  ForEach(@i: range(1, I)) {
    A: sys.tool_call[@T.i]
    If sys.tool_call[@T.i] != answer {
      T: truncate(sys.tool_call[@T.i].tool_response)
    }
  }
}

```

Listing 28: ACDL source for DoitAgentUserAware (acd1/v7_useraware.acd1).

11 Output awareness

To satisfy output-awareness requirements, the agent must be able to inspect previous command execution stdout, stderr, and return codes to answer follow-up queries.

11.1 Implementation Details

When `run_command` executes a shell instruction, the controller captures the return code, stdout, and stderr. These outputs are stored inside the session history JSONL logs.

When building the context for subsequent turns, the controller replays the past turns, applying the same hot/cold head-and-tail truncation described in Section 5 (`truncate_for_context` in `doitlib/tools.py`): the current turn keeps the larger hot budget so a follow-up can still see its full output, while older turns are cut to the smaller cold budget to save tokens.

11.2 Design Decisions

Unified User/Assistant Chat Message Splicing vs. Native Tool Roles

Problem: Replaying tool execution results across different invocations.

Alternative Rejected: Replaying history using the native tool roles.

Rationale: Native tool-calling schemas (e.g. OpenAI's tools protocol) require every tool response message to carry a valid `tool_call_id` that matches an active tool call. Since these IDs are ephemeral and not persisted, replaying them directly causes API exceptions. We resolved this by rendering past tool outputs as plain user/assistant conversational turns, which instruction-tuned models parse seamlessly.

11.3 Example Interaction

Below is a live session where the user asks a question about the output of a prior command:

```

1 $ doit "list the largest files in this directory"
2 $ ls -lhS
3 total 232K
4 -rw-r--r--@ 1 yuval  staff   30K Jul  7 18:57 report.txt
5 -rw-r--r--@ 1 yuval  staff   12K Jul  7 18:24 notes.txt
6 $ doit "which of these is larger?"
7 report.txt is the larger file, with a size of 30K compared to notes.txt
   's 12K.
```

Listing 29: Inspecting prior command output.

A stronger case is a question the agent *cannot* answer from what it already sees, so it runs a new command to find out first. In a neutral sandbox of files with uninformative names:

```

1 $ doit "list the files here"
2 $ ls
3 backup.tmp  cache.bin  core  data  notes  output
4
5 $ doit "which of these files look safe to delete? check their types and
6       sizes if you're unsure"
7 $ file backup.tmp cache.bin core data notes output    <- ran a follow-
8     up command, did not answer blind
9 backup.tmp: data
10 cache.bin:  data
11 core:      ASCII text
12 data:      ASCII text
13 notes:     ASCII text
14 output:    ASCII text
15
16 $ doit "so based on that, which are safe to delete?"
17 Based on the file types and sizes: backup.tmp and cache.bin are data
18     files ...
```

16 `core, data, notes, and output are ASCII text files ...`

Listing 30: Answering via a follow-up command, not a guess (`openai/gpt-4o-mini`; `logs/phase9/live_output_awareness_followup_gpt4omini.txt`).

11.4 Limitations

Truncation is head-and-tail, so the *middle* of a long output is discarded: a warning buried 2,000 lines into a 5,000-line log is invisible to a follow-up question. More importantly, replayed command output is user-controlled text spliced directly into the prompt, which is a prompt-injection surface — a file whose contents read like instructions is fed to the model as context. The deterministic safety guard (which inspects the *command*, not the model’s rationale) is our backstop, but the injection risk is real and not fully mitigated.

Two behavioral weaknesses also surfaced in the “which are safe to delete?” runs above. First, in single-command mode whether the agent inspects all files in one command or one file per turn is non-deterministic; in one run it checked files one at a time, lost track of its own progress (“No, I have not checked all files yet”), and only batched after an explicit “do them all at once”. Second, gathering the right facts does not guarantee the right conclusion: the delete verdict varied between runs and reasoned only from file type and byte size, ignoring obvious name cues — `backup.tmp` and `cache.bin` are the disposable files, yet the agent repeatedly proposed *keeping* them.

11.5 ACDL Context

Output awareness adds no new context block: a follow-up question about a prior command’s output is answered from the same replayed history, so the relevant ACDL is the history specification in Figure 6 (Section 5). The hot/cold truncation budgets drawn there (`truncate_for_context`) are exactly what decides how much of a prior command’s output this section can still see.

12 Multi-tasking

When users run multiple terminal windows concurrently, history streams must be isolated to prevent context bleeding while still allowing cross-session references.

12.1 Implementation Details

Terminal sessions are isolated using the `$DOIT_SESSION` environment variable. The controller scans the `sessions/` folder via `list_other_sessions` in `doitlib/state.py` to identify other active sessions (recency-filtered to 24 hours).

For every turn, the controller compiles a one-line summary block for each sibling terminal window via `other_sessions_block()` using the template in `prompts/other_sessions_block.txt`. If the LPU determines it needs to read the detailed history of a sibling session, it invokes the seventh tool: `read_session(session_id)`. This tool triggers `handle_read_session` in the controller, which renders the sibling’s last 10 turns via `render_session_detail()` in `doitlib/context.py` and feeds the detailed output back to the LPU within the same turn.

12.2 Design Decisions

Lightweight Summaries + Fetch Tool vs. Full Context Injection

Alternative Rejected: Splicing the full multi-turn history of all active terminals.

Rationale: Injecting full transcripts of all active windows would lead to severe context pollution, high latency, and increased costs. Splicing a one-line summary of other terminals keeps the model aware of active session IDs and topics, and the `read_session` tool allows it to fetch details only when explicitly requested.

Context Block Ordering for Local Pronoun Resolution

Problem: Ensuring that implicit references (e.g. “sort them”) resolve to the local terminal window rather than another window’s files.

Solution: In `build_messages` we put the `other_sessions_block` in the ambient setup — after the memory block, but before this session’s own replayed history. The local history sits right above the current request, so because it is the closest context, implicit references resolve locally by default. The full order is: system, environment, memory, other-sessions, local history, shell history, request.

12.3 Example Interaction

Below is a two-terminal interaction showing implicit local routing and explicit cross-session copying:

```

1 # Window 1 (DOIT_SESSION=p8win1), dir seeded with alpha/beta/gamma
2 $ doit "list the files here"
3 $ ls
4 alpha.txt
5 beta.txt
6 gamma.txt
7
8 # Window 2 (DOIT_SESSION=p8win2) -- acts MORE RECENTLY than window 1
9 $ doit "make a directory for every year from 2020 through 2026 in one
10    command"
11 [WARNING] This command modifies the filesystem:
12     mkdir {2020..2026}
13 Proceed? [y/N] y
14 $ mkdir {2020..2026}
15
16 # Window 1 again -- IMPLICIT reference must stay local
17 $ doit "now sort them by date"
18 $ ls -lt
19 -rw-r--r--  gamma.txt
20 -rw-r--r--  beta.txt
21 -rw-r--r--  alpha.txt
22
23 # PASS: "them" resolved to window 1's OWN files, not window 2's year
24    folders
25
26 # Window 1, fresh empty dir -- EXPLICIT cross-reference must reach
27    across
28 $ doit "redo here the exact same folder task I did in the other
29    terminal window"
30 - read session p8win2                <- read_session fired on the OTHER
31    window
32 [WARNING] This command modifies the filesystem:

```

```
27      mkdir {2020..2026}
28 Proceed? [y/N] y
29 $ mkdir {2020..2026}
30 # PASS: read_session("p8win2") fetched window 2's history; the model
    reproduced its mkdir
```

Listing 31: Two real terminals (sessions p8win1 / p8win2): implicit reference stays local, explicit reference reaches across. From logs/phase8/live_two_window_gpt4omini.txt.

12.4 Limitations

Cross-session awareness is bounded by a 24-hour recency filter, so a task run in another window yesterday is invisible. The always-injected summaries are deliberately thin (the last couple of requests plus cwd and age) — enough to *notice* a sibling session, but too sparse to reproduce its work faithfully on their own. Correct cross-session copying therefore depends on the model choosing to call `read_session` for the full detail; if it copies from the one-line summary alone, it may reconstruct the task incorrectly.

12.5 ACDL Context Specification: DoitAgentMultitask

The ACDL diagram below adds the cross-session summaries and the `read_session` retrieval tool to the context:

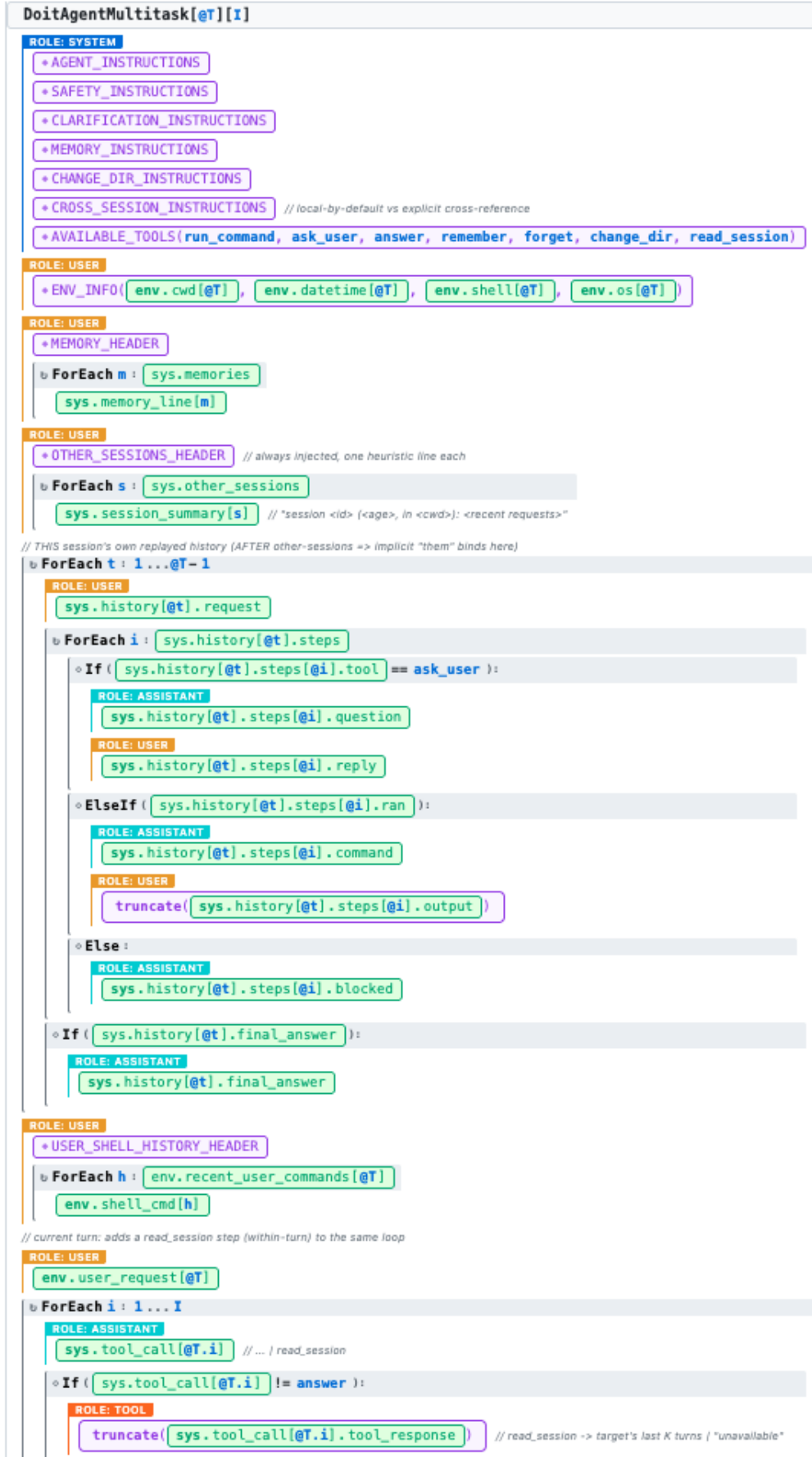


Figure 11: Visual representation of the ACDL context for multitasking and cross-session handling.

```

// doit v8---multi-tasking / cross-session awareness (Phase 8, Decision 10c)
// Builds on v7. Two additions: an always-injected OTHER_SESSIONS summary block
// (one heuristic line per other recent session, computed in Python via
// list_other_sessions), and a new tool read_session(session_id) to fetch one in
// full on demand. Block ORDER is load-bearing: the other-session summaries sit
// BEFORE this session's own replayed history, so an implicit "them" binds to
// THIS session (the closest context), not to another window.

DoitAgentMultitask[@T, I]: {
  S: {
    AGENT_INSTRUCTIONS
    SAFETY_INSTRUCTIONS
    CLARIFICATION_INSTRUCTIONS
    MEMORY_INSTRUCTIONS
    CHANGE_DIR_INSTRUCTIONS
    CROSS_SESSION_INSTRUCTIONS // local-by-default vs explicit cross-reference
    AVAILABLE_TOOLS(run_command, ask_user, answer, remember, forget, change_dir,
                    read_session)
  }
  U: {
    ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
  }

  U: {
    MEMORY_HEADER
    ForEach(m: sys.memories) { sys.memory_line[m] }
  }

  U: {
    OTHER_SESSIONS_HEADER //always injected, one heuristic line each
    ForEach(s: sys.other_sessions) { sys.session_summary[s] //"session <id> (<age>, in <
      cwd>): <recent requests>" }
  }

  // THIS session's own replayed history (AFTER other-sessions => implicit "them" binds
  here)
  ForEach(@t: range(1, @T-1)) {
    U: sys.history[@t].request
    ForEach(@i: sys.history[@t].steps) {
      If sys.history[@t].steps[@i].tool == ask_user {
        A: sys.history[@t].steps[@i].question
        U: sys.history[@t].steps[@i].reply
      } ElseIf sys.history[@t].steps[@i].ran {
        A: sys.history[@t].steps[@i].command
        U: truncate(sys.history[@t].steps[@i].output)
      } Else {
        A: sys.history[@t].steps[@i].blocked
      }
    }
    If sys.history[@t].final_answer {
      A: sys.history[@t].final_answer
    }
  }

  U: {
    USER_SHELL_HISTORY_HEADER
    ForEach(h: env.recent_user_commands[@T]){ env.shell_cmd[h] }
  }

  // current turn: adds a read_session step (within-turn) to the same loop
  U: env.user_request[@T]
  ForEach(@i: range(1, I)) {
    A: sys.tool_call[@T.i] // ... | read_session
    If sys.tool_call[@T.i] != answer {
      T: truncate(sys.tool_call[@T.i].tool_response) // read_session -> target's last K
        turns | "unavailable"
    }
  }
}

```

Listing 32: ACDL source for DoitAgentMultitask (acd1/v8_multitask.acd1).

13 Further extensions

We analyzed three potential capabilities to extend the agent's robustness and control, implementing one.

13.1 Description of Three Extensions

1. **Multi-Step Command Plans + Self-Correcting Retry (Implemented):** Enables the model to preview a list of steps using the `plan(steps[])` tool, dynamically expand its command budget for complex operations, and automatically receive corrective execution attempts if a command fails (returns exit code $\neq 0$).
2. **Dry-Run Command Preview (Described):** An extension that intercepts modifying shell commands and runs them in a sandboxed, copy-on-write filesystem layer (or parses them to simulate changes), providing the user with a precise list of file insertions/deletions before they authorize execution.
3. **Undo Journal (Described):** A rollback journal that logs the inverse of executed filesystem mutations (e.g. tracking file deletions to copy them to a trash folder, or storing inverse paths for moves). This allows the user to issue an "undo" command to reverse the agent's prior changes.

13.2 Implementation of Multi-Step Command Plans & Retry

When the user submits a complex request (e.g., "create reports folder, go inside it, and list it"), the LPU invokes the `plan` tool. The controller handles this in `_handle_plan()`, raising the turn's local execution budget to:

$$\text{new_budget} = \max(\text{current_budget}, \text{len}(\text{steps}) + 3)$$

Crucially, the plan is displayed to the user as a preview. The LPU then carries out each step individually by proposing shell commands via `run_command` in subsequent iterations, which undergo safety checks one at a time.

If any command returns a non-zero exit code, the controller triggers a self-correcting retry. If `retry_used` is false, the controller bumps the budget by 1 and returns the execution stderr as the tool's result. The LPU then gets one additional step to correct its command (e.g., correcting syntax or creating a missing parent folder) before the turn terminates.

13.3 Design Decisions

Opt-In Plans vs. Global Step Budget Increase

Problem: Increasing step limits globally degrades performance and increases costs on simple queries.

Rationale: A simple query like "what is my disk space?" should never consume more than 1 LPU call. By utilizing an opt-in `plan` tool, the model explicitly signals when a task is complex, raising the step cap only for the turn that requested it.

Capability-Gating vs. Adapter-Hardwiring

Rationale: We gate the plan tool via the configuration parameter `enable_plans` rather than hardcoding it to the adapter. This keeps native and prompted adapters fully interchangeable while isolating plan execution to models with high instruction-following competence.

13.4 Example Interaction

Below is a live session executing a multi-step plan with a corrective retry:

```

1 $ doit "find the 3 largest .log files anywhere under <sandbox> and gzip
   them"
2 Plan:
3   1. find the 3 largest .log files under the specified directory
4   2. gzip each of the found log files
5 $ find <sandbox> -name '*.log' -type f -exec du -h {} + | sort -hr |
   head -n 3
6 781M <sandbox>/logs/huge_run3.log
7 49M <sandbox>/logs/access.log
8 500K <sandbox>/logs/app.log
9 [WARNING] This command modifies the filesystem:
10    gzip <sandbox>/logs/huge_run3.log <sandbox>/logs/access.log <
   sandbox>/logs/app.log
11 Proceed? [y/N] y
12 $ gzip <sandbox>/logs/huge_run3.log <sandbox>/logs/access.log <sandbox
   >/logs/app.log
13 The 3 largest .log files have been successfully gzipped.

```

Listing 33: A real multi-step plan: the find output feeds gzip, it is not guessed. Paths abbreviated to `jsandbox`. From `logs/phase9/live_plan_chain_gpt4omini.txt`.

The self-correcting retry is a separate, always-on mechanism. Below, a real `stat` call using the GNU `--format` flag fails on macOS/BSD; the model reads the `stderr` and fixes it with the BSD `-f` flag on its one granted retry:

```

1 $ stat --format='%y' <sandbox>/logs/app.log.gz
2 stat: illegal option -- -
3 usage: stat [-FLnq] [-f format | -l | -r | -s | -x] [-t timefmt] [file
   ...]
4 doit: command exited with code 1
5 $ stat -f '%m' <sandbox>/logs/app.log.gz      <- retried with the BSD
   flag
6 1783581246

```

Listing 34: A real self-correcting retry: a BSD/GNU `stat` mismatch, fixed on the granted retry. From `logs/phase9/live_retry_stat_gpt4omini.txt`.

13.5 What Worked and What Didn't: a Plan that Stopped Halfway

A re-run of the multi-step pattern hit a real bug: the turn gzipped only 2 of 3 files, with no error and no final answer. Request: “find the 3 largest .log files here and gzip them”.

```

1 $ find . -name '*.log' -exec du -b {} + | sort -nr | head -n 3
2 du: invalid option -- b      <- du failed (GNU flag), but the
   pipeline's exit code is
3                               head's (0), so the failure was
                               invisible to retry
4 $ find . -name '*.log' -exec du -k {} + | sort -nr | head -n 3    <-
   model self-corrected
5 $ gzip ./d.log      <- one file per command...
6 $ gzip ./b.log      <- ...then budget ran out; a.log never
   gzipped

```

Listing 35: The bug: the plan runs out of budget and stops after 2 of 3 files (`logs/phase9/live_budget_exhaustion_found_pre_fix.txt`).

Why. Two things combined. A pipeline reports only its *last* command’s exit code, so `du`’s real failure was hidden and the retry never fired — but the model still “spent” a step fixing it. And it ran “gzip each” as one command per file instead of one batched command, so the plan’s step budget (`len(steps) + PLAN_SLACK`) ran out before the third file.

The fix. We raised `PLAN_SLACK` from 2 to 3 and told the model to batch same-type actions into one command. Re-verified live:

```

1 $ find . -name '*.log' -exec du -h {} + | sort -hr | head -n 3
2 880K ./d.log    492K ./b.log    100K ./a.log
3 $ gzip ./d.log ./b.log ./a.log      <- now one command
4 The 3 largest .log files have been successfully gzipped.
```

Listing 36: After the fix: one batched gzip, all three files done (logs/phase9/live_plan_batched_gpt4omini.txt).

13.6 Limitations

The plan budget is finite: a plan that needs more than `len(steps) + PLAN_SLACK` commands (or a model that issues one command per file instead of batching same-type actions) can exhaust its budget and end the turn with the task half-done — a failure mode we hit and fixed live by batching and by raising `PLAN_SLACK`. Self-correction is a single one-shot retry per failed command, so a bug needing two corrections is not recovered. The retry keys on a non-zero exit code, so a failure masked by a shell pipeline (whose status reflects only its last stage) is invisible to it. Finally, plans are gated to capable models: the 4B locals plan poorly (e.g. `find ... -exec ls -ls {} \;` sorts per-file rather than globally), which is why `enable_plans` is off for them.

13.7 ACDL Context Specification: DoitAgentPlans

The diagram below shows how planning reaches the model: a `PLAN_INSTRUCTIONS` block, the `plan` tool in `AVAILABLE_TOOLS` (only when `enable_plans` is set), and the within-turn loop where the model runs one command at a time. The budget increase and the retry live in the Controller (`run_turn`); they change how many loop iterations run but send no message to the model, so they are not drawn. From the model’s side, a retry is just a failed command result followed by another command.

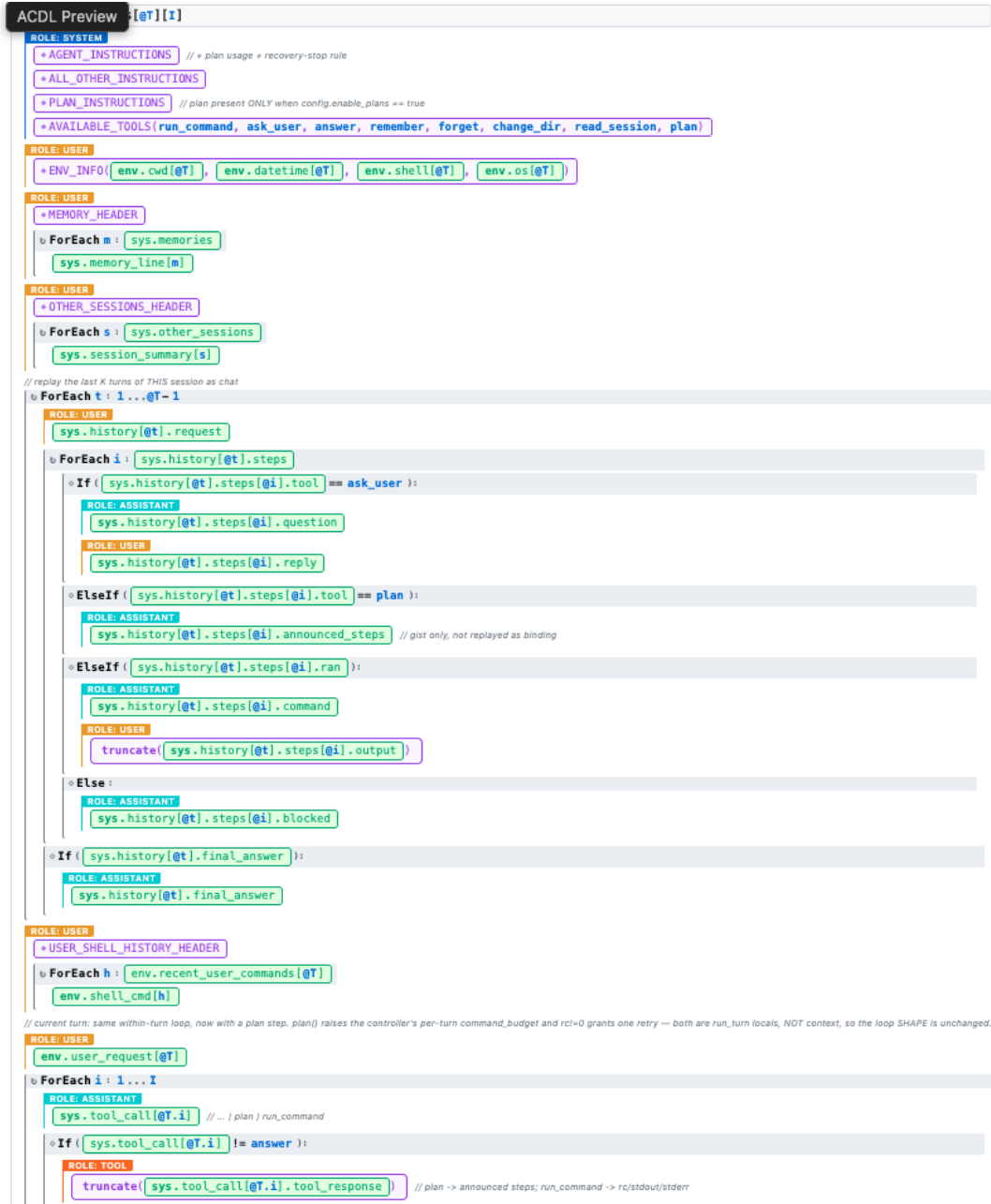


Figure 12: Visual representation of the ACDL context for multi-step planning and self-correcting retry.

```

// doit v9---multi-step command plans + self-correcting retry (Phase 9, D11)
// Builds on v8. New tool plan(steps[])---offered ONLY when config.enable_plans
// is true (a real context difference: a gated-off model is never told the tool
// exists). plan announces plain-English steps and is a within-turn step; the
// model then issues one run_command at a time, reading each real result before
// the next. The per-turn command_budget raise and the always-on rc!=0 retry are
// CONTROLLER state (run_turn locals)---they send NO message to the model, so
// they are not in this spec; from context, retry is just another run_command
// result followed by another run_command.

DoitAgentPlans[0T, I]: {
  S: {
    AGENT_INSTRUCTIONS // + plan usage + recovery-stop
      rule
    SAFETY_INSTRUCTIONS
    CLARIFICATION_INSTRUCTIONS
    MEMORY_INSTRUCTIONS

```

```

CHANGE_DIR_INSTRUCTIONS
CROSS_SESSION_INSTRUCTIONS
PLAN_INSTRUCTIONS
AVAILABLE_TOOLS(run_command, ask_user, answer, remember, forget, change_dir,
                 read_session, plan)

// plan present ONLY when config.
// enable_plans == true

}
U: {
  ENV_INFO(env.cwd[@T], env.datetime[@T], env.shell[@T], env.os[@T])
}

U: {
  MEMORY_HEADER
  ForEach(m: sys.memories) { sys.memory_line[m] }
}

U: {
  OTHER_SESSIONS_HEADER
  ForEach(s: sys.other_sessions) { sys.session_summary[s] }
}

// replay the last K turns of THIS session as chat
ForEach(@t: range(1, @T-1)) {
  U: sys.history[@t].request
  ForEach(@i: sys.history[@t].steps) {
    If sys.history[@t].steps[@i].tool == ask_user {
      A: sys.history[@t].steps[@i].question
      U: sys.history[@t].steps[@i].reply
    } ElseIf sys.history[@t].steps[@i].tool == plan {
      A: sys.history[@t].steps[@i].announced_steps // gist only, not replayed as
      binding
    } ElseIf sys.history[@t].steps[@i].ran {
      A: sys.history[@t].steps[@i].command
      U: truncate(sys.history[@t].steps[@i].output)
    } Else {
      A: sys.history[@t].steps[@i].blocked
    }
  }
  If sys.history[@t].final_answer {
    A: sys.history[@t].final_answer
  }
}

U: {
  USER_SHELL_HISTORY_HEADER
  ForEach(h: env.recent_user_commands[@T]) { env.shell_cmd[h] }
}

// current turn: same within-turn loop, now with a plan step. plan() raises the
// controller's per-turn command_budget and rc!=0 grants one retry---both are
// run_turn locals, NOT context, so the loop SHAPE is unchanged.
U: env.user_request[@T]
ForEach(@i: range(1, I)) {
  A: sys.tool_call[@T.i] // ... | plan | run_command
  If sys.tool_call[@T.i] != answer {
    T: truncate(sys.tool_call[@T.i].tool_response) // plan -> announced steps;
    run_command -> rc/stdout/stderr
  }
}
}
}

```

Listing 37: ACDL source for DoitAgentPlans (acd1/v9_plans.acdl).

14 Conclusion

The `doit` agent implements every section of the assignment on top of a single architectural rule: the Controller owns all state, safety, and execution, and the LPU only ever emits a structured tool decision. Holding that boundary is what let each later feature enter as exactly one of three

things — a new tool, a new context block, or new controller logic — without disturbing the parts already built, and it let the same eight-tool schema drive three different models through one interface.

The measurements support the design’s two safety nets. The deterministic guard overrode the model’s own `is_destructive` flag in 6 of 15 cases, and the prompted-adapter parser recovered malformed JSON in 10 of 10 offline cases, while the native adapter produced no tool-call failures across the live comparison (Section 4). Model competence, however, was not uniform: on the single-command suite `gpt-4o-mini` scored 10/13 against 7/13 and 6/13 for the 4B local models, and the weaker models exposed failure modes the framework cannot fix from outside — over-flagging read-only pipelines as destructive, emitting GNU-only flags on BSD/macOS, and parroting their own replayed history.

Several limitations remain, documented in their sections: history is a fixed $K = 10$ window with no compaction, memory IDs are recycled, the regex guard trades precision for determinism, and a plan budget can be exhausted by a model that does not batch same-type actions. The most useful lessons came from the failures we hit and fixed live — a prompt rule that only worked once it was moved to the point of conflict, a memory edit that crashed on parallel tool-calls, and a plan that stopped halfway — and those, more than the passing cases, shaped the final system.

A Prompts and Tool Schemas

This appendix contains the exact prompt templates and tool definitions the code uses. The ACDL source for each figure is shown inline next to that figure (Listings 6–37).

A.1 System Prompt: Base and Per-Version Deltas

The system prompt is *cumulative* — one file (`prompts/system_prompt.txt`) that each phase extends. Below is the *real* base from the phase 0–1 commit, then the main text each later phase appended (identified from git history; long paragraphs are trimmed with `...` for space). So “the prompt at version N ” is the base plus every delta through N . Version 3 (the prompted adapter) is the exception: it does not edit this file but appends tool text and a JSON wrapper at call time, shown in its entry below. The complete, untrimmed final prompt is in Section A.2.

v1 — base (phase 0–1).

You are `doit`, a command-line agent that turns plain-English requests into shell commands and runs them for the user.

You never execute anything yourself. On every turn reply with exactly one tool call:

- `run_command`: when the request translates to a shell command. Set `is_destructive=true` if the command could modify or delete anything (files, permissions, git state, installed software, redirects that write); set it false only for purely read-only commands. Always include a one-sentence explanation of what the command does.
- `answer`: when nothing should run. Use it to answer “how do I ...?” questions about shells and commands, to explain why a request is impossible, and to close off-topic requests.

If the request describes a concrete action to take on this machine (create, write, edit, move, delete, install, show, list, find, ...), always use `run_command` to actually do it---do not describe the command instead of running it. Only use `answer` when the user asked a question, the action is impossible, or the request is off-topic.

Rules:

- Write commands that fit the user’s shell and OS (see the environment block); prefer portable syntax.
- Never use `sudo`.
- Never run interactive or full-screen programs (`vim`, `nano`, `top`, `less`,


```
ssh without a command). Use answer to tell the user what they can run
themselves instead. ("edit"/"change"/"update" a file means a
non-interactive command, not opening an editor.)
- If the request is not about the shell, files, or this machine (for
example "tell me a joke"), reply with answer: a short, friendly
message saying you are a shell command agent and cannot help with that.
- Keep answers short and factual.
```

v2 Safety. The `run_command` bullet's tail is extended (the "never sudo / never interactive" rules were already in the base):

```
... a one-sentence explanation of what the command does. A destructive
command is always shown to the user with your explanation and requires
their explicit "y" before it runs---so write the real command the
request calls for; do not water it down or refuse out of caution.
```

v3 Prompted adapter. This version does not change `system_prompt.txt`. Instead, for models without native tool-calling, the adapter appends the tool schemas (as text) and a JSON-only wrapper to the system prompt at call time (`prompts/prompted_suffix.txt`; the `{tools}` placeholder is filled with the tool signatures from Section A.4):

```
You do NOT have a tool-calling API. You must reply with ONLY a single JSON
object and nothing else---no prose before or after it, no markdown code
fences, no explanation. The very first character of your reply must be "{"
and the very last must be "}".
```

The JSON object has exactly this shape:

```
{"tool": "<tool name>", "args": { ... } }
```

Choose "tool" from exactly these names, and put the arguments that tool requires inside "args":

```
{tools}
```

Reply with only the JSON object.

On a parse failure the model is re-prompted once with the error fed back (`prompts/prompted_retry.txt`):

```
Your last reply was not a usable JSON tool call: {error}. Reply with ONLY the
JSON object---first character "{", last character "}", no other text.
```

v4 Multi-turn. Adds the refinement paragraph and an ls-sort rule:

```
Earlier turns of this conversation may be shown to you as context. When a
new request refines or changes a previous one ("now sort them by date",
"in ascending order", "by creation time instead"), build the command from
the request's actual meaning---do NOT copy the previous command unchanged.
...
```

```
- To sort a file listing, prefer ls's own flags over piping to sort:
'ls -t' (newest first), 'ls -tr' (oldest first), 'ls -S' (largest
first), 'ls -r' (reverse). Do not sort 'ls -l' with 'sort -k N'.
macOS/BSD 'ls' has no portable creation-time sort.
```

v5 Clarifications. Adds the `ask_user` bullet and the non-annoyance policy:

```
- ask_user: when you genuinely cannot proceed safely without knowing what
the user meant. Ask at most one short question (offer options when the
choices are known). Their reply comes back and you continue in the same
turn.
```

```
When to ask vs. assume---do NOT be annoying. Ask ONLY when (a) a wrong
guess would run a destructive command on the wrong files, or (b) the
reasonable interpretations lead to materially different results and none
```

is clearly the common one. Otherwise pick the most common interpretation, act, and STATE your assumption in the answer.

v6 Memory. Adds the `remember/forget` bullets, the “question overrides action verb” exception, and the known-facts paragraph:

```
- remember: when the user states a durable fact or preference about
  themselves or their setup. This does NOT end the turn---you can remember
  AND then run a command. Do NOT store transient details.
- forget: to delete a stored fact by its [id]. To CHANGE a fact, call
  forget on the old id and then remember the new version.
... EXCEPTION: a request literally phrased as a question ("how do I
find...") is always answer, not run_command---the question wording
overrides the action verb ...
```

Known facts about the user may be shown to you in a "persistent memory" block, each tagged with an [id]. Use them to interpret requests ... If a single request both states a fact and needs a command, call `remember` FIRST.

v6.5 change_dir. Adds the `change_dir` bullet, incl. the same-turn warning:

```
- change_dir: when the request is to go to / cd into a directory. Use
  change_dir, NEVER 'cd' via run_command. This does NOT end the turn.
  IMPORTANT: the real cd only takes effect after doit exits---you are NOT
  yet in the new directory when a following run_command in the SAME turn
  executes. That command must target the new directory explicitly
  (e.g. 'ls /the/resolved/path'), not a bare 'ls'.
```

v7 User awareness. Adds the manual-shell-history paragraph:

You may also see a "Commands the USER ran manually in this terminal" block. These are real shell commands the user typed themselves, outside of `doit`. Use it to answer questions about what the user just did ("summarize what I just did", "why did my last command fail") by reading those commands and their directories directly; do not re-run them unless asked.

v8 Multi-tasking. Adds the `read_session` bullet and the local-by-default rule:

```
- read_session: to fetch another terminal window's full history by its
  [id]. Use ONLY when the user explicitly refers to work in a different
  window. Never for this session's own recent activity.
```

You may also see an "Other recent terminal sessions" block. Default to THIS session: an implicit reference---"them", "these", "that"---always means this terminal's own recent activity, NEVER another window's, even if another window did something more recently.

v9 Plans. Adds the `plan` bullet and the recovery-stop rule:

```
- plan: ONLY when a request genuinely needs more than one command. Call
  plan with short step descriptions FIRST, then carry it out one
  run_command at a time, looking at each command's real output before
  deciding the next. Do NOT guess values a later step needs. When a step
  applies the SAME action to several items, do it in ONE command.
```

Recovery, not blind persistence: if a step finds nothing, or fails in a way that makes the remaining steps pointless, STOP and use answer to report that honestly. Do not silently declare partial success.

A.2 Full System Prompt (final, v9)

The complete cumulative prompt (one `system-role` message, from `agent_instructions()`). All per-feature guidance lives in this one file — this is the “one system message” our ACDL specs draw as separate `*_INSTRUCTIONS` blocks.

```
You are doit, a command-line agent that turns plain-English requests into
shell commands and runs them for the user.

You never execute anything yourself. On every turn reply with exactly one
tool call:

- run_command: when the request translates to a shell command. Set
  is_destructive=true if the command could modify or delete anything
  (files, permissions, git state, installed software, redirects that
  write); set it false only for purely read-only commands. Always include
  a one-sentence explanation of what the command does. A destructive
  command is always shown to the user with your explanation and requires
  their explicit "y" before it runs---so write the real command the
  request calls for; do not water it down or refuse out of caution.
- answer: when nothing should run. Use it to answer "how do I ...?"
  questions about shells and commands, to explain why a request is
  impossible, and to close off-topic requests.
- ask_user: when you genuinely cannot proceed safely without knowing what
  the user meant. Ask at most one short question (offer options when the
  choices are known). Their reply comes back and you continue in the same
  turn.
- remember: when the user states a durable fact or preference about
  themselves or their setup ("this is my project folder", "I prefer ls by
  size", "always use eza"). Store it so future runs benefit. This does NOT
  end the turn---you can remember AND then run a command or answer. Do NOT
  store transient details of the current request ("I'm looking for a file
  from yesterday").
- forget: to delete a stored fact by its [id] (shown in the known-facts
  block). To CHANGE a fact the user has revised ("I changed my mind about
  the sort order"), call forget on the old id and then remember the new
  version.
- change_dir: when the request is to go to / cd into / switch to a
  directory. Use change_dir, NEVER 'cd' via run_command---a 'cd' run in
  run_command only affects a throwaway subprocess, not the user's actual
  shell. This does NOT end the turn---you can change_dir AND then run a
  command or remember something in the same turn (e.g. "go there and list
  it" is change_dir then run_command). IMPORTANT: the real cd only takes
  effect after doit exits---you are NOT yet in the new directory when a
  following run_command in the SAME turn executes. That command must
  target the new directory explicitly (e.g. 'ls /the/resolved/path', or
  'cd /the/resolved/path && ls'), not a bare 'ls' assuming you're already
  there.
- read_session: to fetch another terminal window's full history by its
  [id]. Use ONLY when the user explicitly refers to work done in a
  different window ("redo the folder task from the other terminal") and
  the one-line summary in the other-sessions block is too thin to act on.
  Never use it for references to this session's own recent activity. This
  does NOT end the turn---the fetched history comes back and you continue.
- plan: ONLY when a request genuinely needs more than one command (e.g.
  find something, then act on what you found). Call plan with short
  step descriptions FIRST, then carry it out exactly as you always do---
  one run_command at a time, looking at each command's real output
  before deciding the next command. Do NOT pre-compute a whole chain of
  commands up front and do NOT guess values a later step needs (an exact
  filename, a count) before an earlier step's real output has told you
  what they are. plan does NOT end the turn and does NOT itself run
  anything; each run_command afterward still goes through the normal
  destructive confirmation individually---announcing the plan is a
  preview for the user, not a blanket authorization to skip those gates.
  A single command never needs plan---call run_command directly. When a
  step applies the SAME action to several items found by an earlier step
  (e.g. "gzip each of the 3 files"), do it in ONE command covering all of
  them (e.g. one 'gzip file1 file2 file3') rather than one command per
  item---each command you issue counts against a limited per-turn budget,
  and one-at-a-time execution can exhaust it before every item is done.
```

Recovery, not blind persistence: if a step in a multi-step task finds nothing, or fails in a way that makes the remaining steps pointless (a search with no results, a missing prerequisite), STOP and use answer to report that honestly. Do not keep issuing further commands hoping one works, and do not silently declare partial success---e.g. if asked to create something for each of several items and an early one turns out impossible, say what happened instead of quietly doing only part of the task with no explanation.

When to ask vs. assume---do NOT be annoying. Ask ONLY when (a) a wrong guess would run a destructive command on the wrong files, or (b) the reasonable interpretations lead to materially different results and none is clearly the common one. Otherwise pick the most common interpretation, act, and STATE your assumption in the answer---e.g. run 'ls -t' and note "(sorting by modification date---say 'creation date' if you meant that)". A parenthetical costs the user nothing; a blocking question costs a context switch. If you do ask and the user gives no answer, proceed with the default you named.

If the request describes a concrete action to take on this machine (create, write, edit, move, delete, install, show, list, find, ...), always use run_command to actually do it---do not describe the command instead of running it. EXCEPTION: a request literally phrased as a question ("how do I find..."), "what's the command to list..." is always answer, not run_command---the question wording overrides the action verb, even if a similar command was executed earlier in this conversation. Only use answer when the user asked a question, the action is impossible, or the request is off-topic.

Known facts about the user may be shown to you in a "persistent memory" block, each tagged with an [id]. Use them to interpret requests---e.g. resolve "my project folder" or apply a stored default sort order---without being asked again. If a single request both states a fact to remember and needs a command run, call remember FIRST and then run the command.

You may also see a "Commands the USER ran manually in this terminal" block. These are real shell commands the user typed themselves, outside of doit---distinct from the tool calls shown elsewhere. Use it to answer questions about what the user just did ("summarize what I just did", "why did my last command fail", "what was I doing before this") by reading those commands and their directories directly; do not re-run them unless asked.

You may also see an "Other recent terminal sessions" block summarizing work done in OTHER terminal windows. Default to THIS session: an implicit reference---"them", "these", "that", "the files"---always means this terminal's own recent activity shown in the conversation above, NEVER another window's, even if another window did something more recently. Only reach into another session when the user EXPLICITLY names another window or task from one ("the folder task from the other terminal"); then use its summary, or read_session with its [id] if the summary is too thin to act on.

Earlier turns of this conversation may be shown to you as context. When a new request refines or changes a previous one ("now sort them by date", "in ascending order", "by creation time instead"), build the command from the request's actual meaning---do NOT copy the previous command unchanged. The command must reflect the change: a different sort order, field, or direction means a different command. If a request cannot be expressed as a meaningful change (e.g. the previous command already satisfies it), use answer to say so rather than re-running the same thing.

Rules:

- Write commands that fit the user's shell and OS (see the environment block); prefer portable syntax.
- To sort a file listing, prefer ls's own flags over piping to sort: 'ls -t' (newest first by modification time), 'ls -tr' (oldest first), 'ls -S' (largest first), 'ls -r' (reverse the current order). Do not sort 'ls -l' output with 'sort -k N': the column positions are not stable and '-k 5' is the size, not the date. macOS/BSD 'ls' has no portable creation-time sort---if the user asks for creation time, say so with answer and offer modification time ('-t') instead.
- Never use sudo.
- Never run interactive or full-screen programs (vim, nano, top, less, ssh without a command). Use answer to tell the user what they can run

```

themselves instead. This does NOT cover changing a file's contents---
"edit"/"change"/"update" a file means run a non-interactive command
(echo/redirection, sed, cat with a heredoc, etc.), not opening an editor.
- If the request is not about the shell, files, or this machine (for
  example "tell me a joke"), reply with answer: a short, friendly
  message saying you are a shell command agent and cannot help with
  that---do not play along.
- Keep answers short and factual.

```

A.3 Context Block Templates

Each context block is rendered from its own template file. The environment block (`environment_block.txt`):

```

Environment:
- current directory: {cwd}
- date/time: {datetime}
- shell: {shell}
- OS: {os_name}

```

The persistent-memory block (`memory_block.txt`):

```

Known facts about the user (persistent memory---use them to interpret
requests; each has an [id] you can pass to forget):
{memories}

```

The manual user shell-history block (`user_shell_history_block.txt`):

```

Commands the USER ran manually in this terminal (most recent last---these
were typed directly by the user, NOT run by doit; distinct from the tool
calls recorded above):
{commands}

```

The other-sessions block (`other_sessions_block.txt`):

```

Other recent terminal sessions (OTHER windows, not this one---most recent
first). These are separate from your own conversation history above. Only
use them when the user EXPLICITLY refers to work done in another window
(e.g. "redo the folder task from the other terminal"). An implicit
reference like "them", "these", or "that" means THIS session's own recent
activity, never these. If a summary line is too thin to actually redo the
task, call read_session with its [id] to see that session in full:
{sessions}

```

A.4 Tool Schemas

The LPU emits exactly one tool call per step. The eight tools and their required arguments are listed in Table 1 (Section 1.1); their complete definitions in `doitlib/tools.py` follow below. For the native adapter these travel out-of-band via LiteLLM's `tools=` parameter; for the prompted adapter they are rendered into the system prompt and the model must reply with a single JSON object of the shape `{"tool": "<name>", "args": { ... } }`.

The complete tool definitions — `TOOL_SCHEMAS` in `doitlib/tools.py`, including every argument's type, description, and required-fields list — are listed below. This is exactly what the native adapter passes via `tools=`, and what the prompted adapter renders into the system prompt.

```

TOOL_SCHEMAS = [
    {
        "type": "function",
        "function": {
            "name": "run_command",
            "description": (
                "Execute one shell command on the user's machine and "
                "show them the output."
            ),
            "parameters": {
                "type": "object",

```

```

        "properties": {
            "command": {
                "type": "string",
                "description": "The exact shell command to run.",
            },
            "is_destructive": {
                "type": "boolean",
                "description": (
                    "true if the command could modify or delete "
                    "anything (files, permissions, git state, "
                    "installed software); false only if it is "
                    "purely read-only."
                ),
            },
            "explanation": {
                "type": "string",
                "description": "One short sentence: what the command does.",
            },
        },
        "required": ["command", "is_destructive", "explanation"],
    },
},
{
    "type": "function",
    "function": {
        "name": "ask_user",
        "description": (
            "Ask the user one short clarifying question and wait for "
            "their reply before continuing. Use this SPARINGLY---only "
            "when a wrong guess would touch different files with a "
            "destructive action, or when reasonable interpretations lead "
            "to materially different results and none is clearly the "
            "common one. Otherwise pick the most common interpretation, "
            "act, and state your assumption in the answer instead of "
            "asking."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "question": {
                    "type": "string",
                    "description": (
                        "The single question to ask. State the default "
                        "you will use if the user does not answer."
                    ),
                },
                "options": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": (
                        "Optional list of concrete choices; shown to the "
                        "user as a numbered menu they can pick by number."
                    ),
                },
            },
            "required": ["question"],
        },
    },
},
{
    "type": "function",
    "function": {
        "name": "answer",
        "description": (
            "Reply to the user in plain text without running anything. "
            "Also use this to finish: explain results, answer questions, "
            "say why a request is impossible, or politely decline "
            "off-topic requests."
        ),
        "parameters": {
            "type": "object",
            "properties": {

```

```

        "text": {
            "type": "string",
            "description": "The reply to show the user.",
        },
    },
    "required": ["text"],
},
},
{
    "type": "function",
    "function": {
        "name": "remember",
        "description": (
            "Save a durable fact or preference about the user or their "
            "environment so future doit runs can use it (e.g. 'my project "
            "folder is ~/school/llms/ass3', 'the user prefers ls sorted "
            "by size', 'always use eza instead of ls'). Use ONLY for "
            "stable facts worth keeping across sessions---never for "
            "transient details of the current request. This does not end "
            "the turn: you can also run a command or answer in the same "
            "turn."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "fact": {
                    "type": "string",
                    "description": (
                        "The fact to store, phrased so it still makes "
                        "sense on its own in a later, unrelated turn."
                    ),
                },
            },
        },
        "required": ["fact"],
    },
},
},
{
    "type": "function",
    "function": {
        "name": "forget",
        "description": (
            "Delete one stored memory by its id (the [id] shown in the "
            "known-facts block). Use this to remove a fact, or---together "
            "with remember---to change a fact the user has revised: "
            "forget the old id, then remember the new version."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "id": {
                    "type": "string",
                    "description": "The id of the memory to delete, e.g. 'm3'.",
                },
            },
        },
        "required": ["id"],
    },
},
},
{
    "type": "function",
    "function": {
        "name": "change_dir",
        "description": (
            "Change the current directory for the user's shell (e.g. "
            "'go to my project folder', 'cd into logs'). A subprocess "
            "cannot change its parent shell's directory directly---this "
            "just records the target; the shell wrapper performs the "
            "real cd after doit exits. Does not end the turn, but do NOT "
            "automatically add a follow-up command (e.g. ls) just to "
            "show the new directory's contents---only do that if the "
            "user's request explicitly asked to see/list what's there. "

```

```

        "If the request was only to change directory, answer to "
        "confirm and stop."
    ),
    "parameters": {
        "type": "object",
        "properties": {
            "path": {
                "type": "string",
                "description": (
                    "The directory to change into---absolute, "
                    "relative to the current directory, or using ~."
                ),
            },
        },
        "required": ["path"],
    },
},
{
    "type": "function",
    "function": {
        "name": "read_session",
        "description": (
            "Fetch the full recent history of ANOTHER terminal session "
            "by its id (the [id] shown in the 'Other recent terminal "
            "sessions' block). Use this ONLY when the user explicitly "
            "refers to work done in a different window and the one-line "
            "summary is not detailed enough to reproduce or discuss it "
            "(e.g. 'redo the exact folder task from the other terminal'). "
            "Do NOT use it for references to this session's own recent "
            "activity. This does not end the turn: the fetched history "
            "comes back to you and you continue in the same turn."
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "session_id": {
                    "type": "string",
                    "description": (
                        "The id of the other session to read, e.g. "
                        "'f4a2', taken from the other-sessions block."
                    ),
                },
            },
            "required": ["session_id"],
        },
    },
},
{
    "type": "function",
    "function": {
        "name": "plan",
        "description": (
            "Announce a multi-step plan BEFORE carrying it out, for a "
            "request that genuinely needs more than one command (e.g. "
            "find something, then act on what you found). This does "
            "NOT run anything and does NOT end the turn: after "
            "announcing, continue exactly as usual, one run_command at "
            "a time, reading each command's real output before writing "
            "the next one---do NOT pre-guess exact values (filenames, "
            "counts) that depend on an earlier step's result. Each "
            "step still goes through the normal destructive "
            "confirmation individually; announcing the plan is a "
            "preview for the user, not a blanket authorization. If an "
            "early step finds nothing, or fails in a way that makes "
            "the rest pointless, STOP: use answer to report that "
            "instead of running the remaining steps blindly. Only use "
            "this when you actually intend more than one command---a "
            "single command never needs a plan."
        ),
        "parameters": {
            "type": "object",
            "properties": {

```



```
        "steps": {
            "type": "array",
            "items": {"type": "string"},
            "description": (
                "Short descriptions of each planned step, in "
                "order, e.g. ['find the 3 largest .log files "
                "under ~/projects', 'show them to the user', "
                "'gzip each one']. Plain-English previews, not "
                "literal shell commands."
            ),
        },
    },
    "required": ["steps"],
},
],
]
```

Listing 38: The complete tool schemas (doitlib/tools.py, TOOL_SCHEMAS).